LAPORAN TUGAS BESAR 1

IF3170 INTELIGENSI ARTIFISIAL



Disusun oleh:

Muhammad Naufal Romi Annafi - 13524058

Mahmudia Kimdaro Amin - 13524083

Eduard Daniel Ariajaya - 13524129

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

1. Deskripsi Persoalan

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti memaksimalkan fungsi objektif.

Daftar Variabel Permasalahan

Entitas	**Atribut**	**Contoh**	**Keterangan**
Package	id	“RusdiPackage”	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar (seluruh bagian paket di luar)
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (*initial state*) random.
- Langkah neighbor (*move*) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
    3. Merotasi paket 90° terhadap sumbu x, y, atau z
- Representasi solusi (final state) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. Value pada state tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- Objective function

  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.

- Kardus tidak bisa melayang dan menembus dinding peti kemas

  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\})$$

Sebuah titik di alas memiliki *support* apabila:

1. $Z_{i=0}$, artinya di alas truk, atau
2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- Barang pecah-belah tidak boleh di bawah paket lain

  Barang pecah-belah tidak boleh menjadi *support*

- Truk tidak boleh mengangkut melebihi kapasitasnya

Berikut merupakan hal-hal yang perlu dilakukan:

Implementasikan 3 algoritma *local search* dengan rincian sebagai berikut:

1. Salah satu algoritma Hill-climbing
2. Simulated Annealing
3. Genetic Algorithm

Menjalankan setiap algoritma (selain Genetic Algorithm) sebanyak 3 kali, kemudian catat beberapa hal berikut:

1. Berlaku untuk semua algoritma
   a. State awal dan akhir
   b. Nilai objective function akhir yang dicapai
   c. Plot nilai objective function terhadap banyak iterasi yang telah dilewati
   d. Durasi proses pencarian
2. Berlaku hanya untuk Steepest Ascent Hill-Climbing dan Stochastic Hill-climbing
   a. Banyak iterasi hingga proses pencarian berhenti
3. Berlaku hanya untuk Hill-Climbing with Sideways Move
   a. Banyak iterasi hingga proses pencarian berhenti
   b. Tambahkan parameter maximum sideways move, dimana ketika banyak sideways move yang dilakukan sudah mencapai maksimum, pencarian dihentikan
4. Berlaku hanya untuk Random Restart Hill-climbing
   a. Banyak restart
   b. Banyak iterasi per restart

    c. Tambahkan parameter maximum restart, dimana ketika banyak restart sudah mencapai maksimum, pencarian dihentikan.
5. Berlaku hanya untuk Simulated Annealing
    a. Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati
    b. Frekuensi '*stuck*' di *local optima*

Khusus untuk Genetic Algorithm, dilakukan beberapa hal berikut:

1. Terdapat 2 parameter yang dapat diubah, yaitu jumlah populasi dan banyak iterasi.
2. Jadikan jumlah populasi sebagai kontrol, kemudian pilih 3 variasi banyak iterasi yang berbeda. Jalankan program sebanyak masing-masing 3 kali untuk setiap konfigurasi parameter.
3. Jadikan banyak iterasi sebagai kontrol, kemudian pilih 3 variasi jumlah populasi yang berbeda. Jalankan program sebanyak masing-masing 3 kali untuk setiap konfigurasi parameter.
4. Untuk setiap eksperimen, catat beberapa hal berikut:
    a. State awal dan akhir
    b. Nilai objective function akhir yang dicapai
    c. Plot nilai objective function terhadap banyak iterasi yang telah dilewati (Cukup plot nilai objective function maksimum dan rata-rata dari populasi terhadap banyak iterasi yang telah dilewati.)
    d. gunakan #386641
    e. Jumlah populasi
    f. Banyak iterasi
    g. Durasi proses pencarian

Lakukan analisis terhadap hasil eksperimen, berikut merupakan beberapa pertanyaan yang menjadi acuan analisis umum:

1. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?
2. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?
3. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

Berikut merupakan pertanyaan yang menjadi acuan analisis dari hill climbing:

1. Seberapa besar nilai objective meningkat dari state awal ke state akhir pada tiap kasus uji?

2. Bagaimana perkembangan nilai objective hingga pencarian berhenti?

Berikut merupakan pertanyaan yang menjadi acuan analisis dari simulated annealing:

1. Seberapa besar nilai objective meningkat dari state awal ke state akhir pada tiap kasus uji?
2. Bagaimana perkembangan nilai objective hingga pencarian berhenti?
3. Bagaimana durasi dari algoritma terpengaruh oleh temperatur?

Berikut merupakan pertanyaan yang menjadi acuan analisis dari genetic algorithm:

1. Bagaimana pengaruh banyak iterasi terhadap nilai objective akhir dan durasi pencarian ketika jumlah populasi tetap?
2. Bagaimana pengaruh jumlah populasi terhadap nilai objective akhir dan durasi pencarian ketika banyak iterasi tetap?

2. Pembahasan

2.1. Model dan Operasi Dasar

Entitas dan variabel dasar yang diperlukan untuk merepresentasikan permasalahan utama dalam tugas besar diimplentasikan dalam file models.py.

Classs-nya meliputi

- Orientation

  Representasi enam kemungkinan arah paket. Misalnya XYZ memakai orientasi width, length, height, sedangkan XZY menukar length dan height.

- Package

  Data satu paket yaitu ID, ukuran, nilai, berat, fragile, dan eta.

- Truck

  Ukuran ruang truk dan batas total berat.

- Coordinate

  Posisi (x, y, z) paket di truk.

- Placement

  Representasi koordinat dan orientasi paket dalam truk. None jika paket tidak dimuat di truk

- State

  Kumpulan placement dari paket yang merepresentasikan calon solusi permasalahan.

- Problem

  Menyimpan kondisi soal yang ingin diselesaikan yaitu truk yang tersedia dan daftar paket.

Selain itu, ada kumpulan operasi dasar yang diimplementasikan dalam file core.py yaitu

- oriented_dimension()

menghasilkan ukuran (lebar, panjang, tinggi) paket setelah diputar sesuai Orientation.

- objective()

  memiliki dua opsi untuk menghitung nilai objektif yaitu 1) langsung menambah semua value paket yg ada di truk 2) dikombinasikan menggunakan nilai ETA menggunakan rumus value (1+ 1/(1+eta)) dengan eta yang makin kecil akan menghasilkan nilai objektif yang makin bagus.

```
def objective(problem: Problem, state: State) -> float:
    return sum(
        package_objective_value(problem,
problem.packages[placement.package_id])
        for placement in state.placements.values()
        if placement.position is not None
    )

def package_objective_value(problem: Problem, package: Package)
-> float:
    if problem.objective_mode == "eta_priority":
        return package.value * (1 + 1 / (1 + package.eta))
    return package.value
```

- total_weight()

  menjumlahkan berat paket yang dimuat.

- is_valid()

  mengembalikan True jika State tersusun benar: data penempatan sesuai, paket berada di dalam truk, tidak bertabrakan, berat tidak melebihi kapasitas, dan paket yang ditumpuk punya penopang.

- random_initial_state()

  Membuat susunan awal secara acak. Paket dicoba diletakkan di lantai truk. Jika penempatan sebuah paket tidak valid, paket tersebut dibiarkan tidak dimuat. Alur fungsinya meliputi:

  1. Semua paket awalnya dianggap di luar truk.
  2. Urutan paket yg dimasukkan diacak.

3. Untuk setiap paket, ada peluang 50% untuk dilewati. Jika tidak dilewati, paket akan dimasukkan truk lalu di-random orientasi x dan y-nya namun z-nya 0 karena berada di lantai.
4. Setiap susunan baru akan diperiksa dengan is_valid(). Jika tidak valid, paket akan dikembalikan ke luar truk.

```
def random_initial_state(problem: Problem, rng: random.Random)
-> State:
    placements = {
        package_id: Placement(package_id, None, Orientation.XYZ)
        for package_id in problem.packages
    }
    package_ids = list(problem.packages)
    rng.shuffle(package_ids)

    for package_id in package_ids:
        if rng.random() < 0.5:
            continue

        package = problem.packages[package_id]
        orientation = rng.choice(list(Orientation))
        width, length, height = oriented_dimensions(package,
orientation)
        truck = problem.truck

        if width > truck.width or length > truck.length or height
> truck.height:
            continue

        placement = Placement(
            package_id,
            Coordinate(
                rng.randint(0, truck.width - width),
                rng.randint(0, truck.length - length),
                0,
            ),
            orientation,
        )
        placements[package_id] = placement
        state = State(placements)

        if not is_valid(problem, state):
            placements[package_id] = Placement(package_id, None,
Orientation.XYZ)
    return State(placements)
```

- generate_valid_neighbor()

  Mencoba perubahan acak pada satu State yaitu

  - move: pilih satu paket dengan paket bisa dikeluarkan dari truk (peluang 25%) atau dipindahkan ke koordinat acak, termasuk kemungkinan posisi bertumpuk pada arah z.
  - rotate: pilih satu paket dan putar orientasinya pada salah satu sumbu.
  - swap: pilih dua paket dan tukar posisi keduanya.

  perubahan posisi paket dilakukan hingga menemukan satu susunan baru yang valid. Mengembalikan None bila tidak menemukannya dalam batas 100 kali percobaan.

```python
def generate_valid_neighbor(
    problem: Problem,
    state: State,
    rng: random.Random,
    max_attempts: int = 100,
) -> State | None:
    if not state.placements:
        return deepcopy(state)

    moves = ["move", "rotate"]
    if len(state.placements) >= 2:
        moves.append("swap")

    for _ in range(max_attempts):
        move = rng.choice(moves)
        if move == "move":
            candidate = _move_neighbor(problem, state, rng)
        elif move == "rotate":
            candidate = _rotate_neighbor(state, rng)
        else:
            candidate = _swap_neighbor(state, rng)

        if (
            _state_key(candidate) != _state_key(state)
            and is_valid(problem, candidate)
        ):
            return candidate

    return None
```

- generate_valid_neighbors()

  Menghasilkan semua susunan tetangga valid yang berbeda melalui perubahan posisi, rotasi, atau pertukaran posisi. Alurnya meliputi:

  1. Periksa state asal. Jika tidak valid, fungsi melempar ValueError.
  2. Untuk setiap paket, coba ubah posisinya. Paket dicoba di luar truk (None) dan di setiap koordinat yang mungkin di dalam truk. Jadi paket di luar bisa dimasukkan, sedangkan paket di dalam bisa dipindahkan atau dikeluarkan.
  3. Untuk setiap paket, coba putar orientasinya pada sumbu x, y, dan z.
  4. Untuk setiap pasangan paket, coba tukar posisi keduanya.

```
def generate_valid_neighbors(problem: Problem, state: State) ->
Iterator[State]:
    if not is_valid(problem, state):
        raise ValueError("Cannot generate neighbors from an
invalid state")

    seen = {_state_key(state)}

    for package_id, placement in state.placements.items():
        package = problem.packages[package_id]
        width, length, height = oriented_dimensions(package,
placement.orientation)
        truck = problem.truck

        if width <= truck.width and length <= truck.length and
height <= truck.height:
            positions = chain(
                (None,),
                (
                    Coordinate(x, y, z)
                    for x in range(truck.width - width + 1)
                    for y in range(truck.length - length + 1)
                    for z in range(truck.height - height + 1)
                ),
            )
        else:
            positions = iter([None])

        for position in positions:
            if position == placement.position:
```

```
                continue
            candidate = deepcopy(state)
            candidate.placements[package_id].position = position
            if _is_new_valid_neighbor(problem, candidate, seen):
                yield candidate

        rotated_orientations = []
        for axis in ("x", "y", "z"):
            orientation =
_rotate_orientation(placement.orientation, axis)
            if orientation not in rotated_orientations:
                rotated_orientations.append(orientation)

        for orientation in rotated_orientations:
            if orientation == placement.orientation:
                continue
            candidate = deepcopy(state)
            candidate.placements[package_id].orientation =
orientation
            if _is_new_valid_neighbor(problem, candidate, seen):
                yield candidate

    package_ids = list(state.placements)
    for first_index, first_id in enumerate(package_ids):
        for second_id in package_ids[first_index + 1 :]:
            first_position = state.placements[first_id].position
            second_position =
state.placements[second_id].position
            if first_position == second_position:
                continue

            candidate = deepcopy(state)
            candidate.placements[first_id].position =
second_position
            candidate.placements[second_id].position =
first_position
            if _is_new_valid_neighbor(problem, candidate, seen):
                yield candidate
```

Fungsi-fungsi lain merupakan fungsi internal yang berguna untuk memodularkan operasi fungsi dasar menjadi langkah yang lebih kecil sehingga kode lebih mudah dibaca dan dipakai ulang.

2.2. Hill Climbing

Implementasi Hill Climbing pada tugas ini mengacu pada konsep local search yang dijelaskan dalam materi Beyond Classical Search IF3170. Algoritma ini melakukan pencarian dengan memulai dari sebuah state awal yang dibangkitkan secara acak, kemudian berpindah menuju state tetangga yang memiliki nilai objective function lebih tinggi.

2.2.1. Steepest Ascent Hill-Climbing

Steepest Ascent selalu memilih tetangga valid dengan objective terbesar yang lebih tinggi daripada state saat ini. Proses berhenti ketika _best_neighbor() tidak menemukan tetangga yang lebih baik. Kondisi tersebut berarti pencarian telah mencapai local optimum.

```
def steepest_ascent_hill_climbing(
    problem: Problem,
    initial_state: State | None = None,
    *,
    rng: random.Random | None = None,
) -> HillClimbingResult:
    generator = rng or random.Random()
    start_state = _prepare_initial_state(problem,
initial_state, generator)

    started_at = perf_counter()
    run = _steepest_run(problem, start_state,
generator)
    duration = perf_counter() - started_at

    return _to_result("steepest_ascent", run, duration)


def _steepest_run(
    problem: Problem,
    initial_state: State,
    rng: random.Random,
) -> _RunResult:
    current = deepcopy(initial_state)
    current_value = objective(problem, current)
```

```
    history = [current_value]
    iterations = 0
    termination_reason = "local_optimum"

    while True:
        neighbor, neighbor_value = _best_neighbor(
            problem, current, current_value, rng,
allow_equal=False
        )
        if neighbor is None:
            break

        current = neighbor
        current_value = neighbor_value
        history.append(current_value)
        iterations += 1

    return _RunResult(
        deepcopy(initial_state),
        deepcopy(current),
        tuple(history),
        iterations,
        termination_reason,
    )
```

Proses utama berjalan pada fungsi steepest_ascent_hill_climbing() dengan bantuan fungsi _steepest_run() untuk menjalankan pencarian dan _best_neighbor() untuk menentukan tetangga terbaik. Alurnya adalah sebagai berikut:

1. Buat state awal secara acak menggunakan _prepare_initial_state() apabila pengguna tidak memberikan initial state. State menyimpan posisi dan orientasi seluruh paket, termasuk paket yang berada di luar truk. Susunan paket harus memenuhi seluruh constraint.
2. Hitung nilai objective function dari state awal menggunakan objective(). Simpan nilai tersebut sebagai current_value dan masukkan ke dalam history.
3. Bangkitkan seluruh tetangga valid dan pilih tetangga dengan objective function tertinggi menggunakan _best_neighbor()

dengan parameter allow_equal=False. Tetangga hanya dipilih apabila nilainya lebih tinggi daripada current state.

4. Apabila ditemukan tetangga yang lebih baik, pindahkan current state ke tetangga tersebut. Perbarui current_value, tambahkan nilainya ke history, dan tingkatkan jumlah iterasi.
5. Ulangi langkah 3–4 sampai _best_neighbor() tidak menemukan tetangga yang lebih baik. Kondisi ini menandakan pencarian mencapai local optimum.
6. Kembalikan hasil pencarian melalui HillClimbingResult, yang memuat state awal, state akhir, nilai objective akhir, riwayat objective, jumlah iterasi, durasi pencarian, dan alasan terminasi.

2.2.2. Hill-Climbing with Sideways Move

Varian hill-climbing ini memperbolehkan perpindahan ke neighbor dengan objective yang sama. Sideways move berguna untuk melewati kumpulan state bertetangga dengan nilai objective sama. Peningkatan objective tetap diprioritaskan karena fungsi _best_neighbor() memilih nilai tetangga tertinggi.

Tanpa pembatasan, perpindahan bernilai sama dapat membentuk siklus. Variabel consecutive_sideways menghitung sideways move berturut-turut. Nilainya dikembalikan ke nol ketika ditemukan peningkatan. Pencarian berhenti ketika batas max_sideways_moves tercapai atau tidak ada tetangga dengan nilai yang sama maupun lebih tinggi.

```
def sideways_hill_climbing(
    problem: Problem,
    initial_state: State | None = None,
    *,
    max_sideways_moves: int,
    rng: random.Random | None = None,
) -> HillClimbingResult:
    _validate_maximum("max_sideways_moves",
max_sideways_moves)
    generator = rng or random.Random()
    start_state = _prepare_initial_state(problem,
initial_state, generator)
```

```
    started_at = perf_counter()
    current = deepcopy(start_state)
    current_value = objective(problem, current)
    history = [current_value]
    iterations = 0
    consecutive_sideways = 0
    total_sideways = 0
    termination_reason = "local_optimum"

    while True:
        neighbor, neighbor_value = _best_neighbor(
            problem, current, current_value, generator,
allow_equal=True
        )
        if neighbor is None:
            termination_reason = "local_optimum"
            break

        if _is_equal(neighbor_value, current_value):
            if consecutive_sideways >=
max_sideways_moves:
                termination_reason =
"max_sideways_moves"
                break
            consecutive_sideways += 1
            total_sideways += 1
        else:
            consecutive_sideways = 0

        current = neighbor
        current_value = neighbor_value
        history.append(current_value)
        iterations += 1

    duration = perf_counter() - started_at
    run = _RunResult(
        deepcopy(start_state),
        deepcopy(current),
        tuple(history),
```

```
        iterations,
        termination_reason,
        total_sideways,
    )
    return _to_result("sideways", run, duration)
```

Proses utama berjalan pada fungsi sideways_hill_climbing(). Varian ini memperbolehkan perpindahan menuju tetangga dengan nilai objective function yang sama untuk membantu melewati plateau. Parameter max_sideways_moves digunakan untuk membatasi banyaknya sideways move secara berturut-turut. Alurnya adalah sebagai berikut:

1. Buat state awal menggunakan _prepare_initial_state(), kemudian hitung objective function awal. Inisialisasi variabel consecutive_sideways dan total_sideways dengan nilai nol.
2. Pilih tetangga terbaik menggunakan _best_neighbor() dengan parameter allow_equal=True, sehingga tetangga dengan nilai objective yang sama tetap dapat dipilih.
3. Apabila tetangga memiliki nilai objective lebih tinggi, pindahkan current state ke tetangga tersebut dan reset consecutive_sideways menjadi nol.
4. Apabila tetangga memiliki nilai objective yang sama, periksa apakah consecutive_sideways sudah mencapai max_sideways_moves. Jika belum, lakukan perpindahan dan tambahkan nilai consecutive_sideways serta total_sideways sebanyak satu.
5. Ulangi pencarian hingga tidak ditemukan tetangga dengan nilai objective yang sama atau lebih tinggi, atau batas maksimum sideways move berturut-turut telah tercapai.
6. Kembalikan hasil pencarian melalui HillClimbingResult, termasuk state awal dan akhir, riwayat objective, jumlah iterasi, durasi, total sideways move, serta alasan terminasi.

2.2.3. Stochastic Hill-climbing

Stochastic Hill Climbing tidak membangkitkan seluruh neighbor. Pada setiap iterasi, fungsi generate_valid_neighbor() mengambil satu tetangga valid secara acak. Tetangga tersebut hanya diterima apabila objective-nya lebih tinggi daripada state saat ini. Tetangga yang lebih

buruk atau bernilai sama ditolak sehingga state tetap tidak berubah pada iterasi tersebut.

Karena tetangga dipilih secara acak, kegagalan memperoleh perbaikan dalam satu iterasi belum membuktikan bahwa tidak ada tetangga yang lebih baik. Oleh karena itu, max_iterations digunakan sebagai stopping condition. Nilai objective tetap dicatat pada setiap percobaan, termasuk ketika tetangga ditolak.

```
def stochastic_hill_climbing(
    problem: Problem,
    initial_state: State | None = None,
    *,
    max_iterations: int = 1_000,
    rng: random.Random | None = None,
) -> HillClimbingResult:
    _validate_maximum("max_iterations", max_iterations)
    generator = rng or random.Random()
    start_state = _prepare_initial_state(problem,
initial_state, generator)

    started_at = perf_counter()
    current = deepcopy(start_state)
    current_value = objective(problem, current)
    history = [current_value]
    iterations = 0
    termination_reason = "max_iterations"

    while iterations < max_iterations:
        neighbor = generate_valid_neighbor(problem,
current, generator)
        if neighbor is not None:
            neighbor_value = objective(problem,
neighbor)
            if _is_greater(neighbor_value,
current_value):
                current = neighbor
                current_value = neighbor_value

        history.append(current_value)
```

```
        iterations += 1

    duration = perf_counter() - started_at
    run = _RunResult(
        deepcopy(start_state),
        deepcopy(current),
        tuple(history),
        iterations,
        termination_reason,
    )
    return _to_result("stochastic", run, duration)
```

Proses utama berjalan pada fungsi stochastic_hill_climbing(). Parameter max_iterations menentukan batas jumlah iterasi, sedangkan rng digunakan untuk pembangkitan state secara acak. Alurnya adalah sebagai berikut:

1. Buat sebuah state awal menggunakan _prepare_initial_state(), kemudian hitung nilai objective function awal dan simpan ke dalam history.
2. Bangkitkan satu state tetangga secara acak menggunakan generate_valid_neighbor(). Berbeda dengan Steepest Ascent, algoritma ini hanya mengevaluasi satu tetangga pada setiap iterasi.
3. Jika tetangga berhasil dibangkitkan, hitung nilai objective function dari tetangga tersebut.
4. Apabila nilai objective tetangga lebih tinggi daripada current state, lakukan perpindahan menuju tetangga tersebut. Jika nilainya lebih rendah atau sama, pertahankan current state.
5. Catat nilai objective function pada history dan tingkatkan jumlah iterasi, termasuk ketika tidak terjadi perpindahan state.
6. Ulangi langkah 2–5 hingga jumlah iterasi mencapai max_iterations.
7. Kembalikan hasil pencarian melalui HillClimbingResult, yang memuat state awal, state akhir, objective akhir, riwayat objective, jumlah iterasi, durasi pencarian, dan alasan terminasi.

2.2.4. Random Restart Hill-Climbing

Random restart hill-climbing menjalankan steepest ascent beberapa kali dari initial state yang berbeda. Run pertama menggunakan state yang diberikan pemanggil atau state acak hasil _prepare_initial_state(). Run berikutnya menggunakan state baru dari random_initial_state().

Setiap run berhenti secara alami ketika mencapai local optimum. State akhir dengan objective terbesar disimpan sebagai hasil keseluruhan. Pencarian dapat berhenti lebih awal apabila nilainya sama dengan jumlah value seluruh paket. Nilai tersebut merupakan upper bound karena objective tidak mungkin melebihi jumlah value seluruh paket.

Riwayat setiap run digabungkan ke objective_history. Posisi awal setiap run disimpan dalam restart_boundaries agar batas restart dapat ditampilkan pada plot.

```
def random_restart_hill_climbing(
    problem: Problem,
    initial_state: State | None = None,
    *,
    max_restarts: int,
    rng: random.Random | None = None,
) -> HillClimbingResult:
    _validate_maximum("max_restarts", max_restarts)
    generator = rng or random.Random()
    first_state = _prepare_initial_state(problem,
initial_state, generator)

    started_at = perf_counter()
    runs: list[_RunResult] = []
    best_state: State | None = None
    best_value = float("-inf")
    optimum_upper_bound = sum(package.value for package
in problem.packages.values())

    for run_index in range(max_restarts + 1):
        run_state = (
            first_state
            if run_index == 0
            else random_initial_state(problem,
generator)
```

```
        )
        run = _steepest_run(problem, run_state,
generator)
        runs.append(run)
        run_value = objective(problem, run.final_state)

        if best_state is None or _is_greater(run_value,
best_value):
            best_state = deepcopy(run.final_state)
            best_value = run_value

        if _is_equal(best_value, optimum_upper_bound):
            break

    duration = perf_counter() - started_at
    assert best_state is not None

    history: list[float] = []
    boundaries: list[int] = []
    for run in runs:
        boundaries.append(len(history))
        history.extend(run.history)

    reached_upper_bound = _is_equal(best_value,
optimum_upper_bound)
    return HillClimbingResult(
        variant="random_restart",
        initial_state=deepcopy(first_state),
        final_state=best_state,
        final_value=best_value,
        objective_history=tuple(history),
        iterations=sum(run.iterations for run in runs),
        duration_seconds=duration,
        termination_reason=(
            "global_upper_bound" if reached_upper_bound
else "max_restarts"
        ),
        restarts=len(runs) - 1,
        iterations_per_restart=tuple(run.iterations for
```

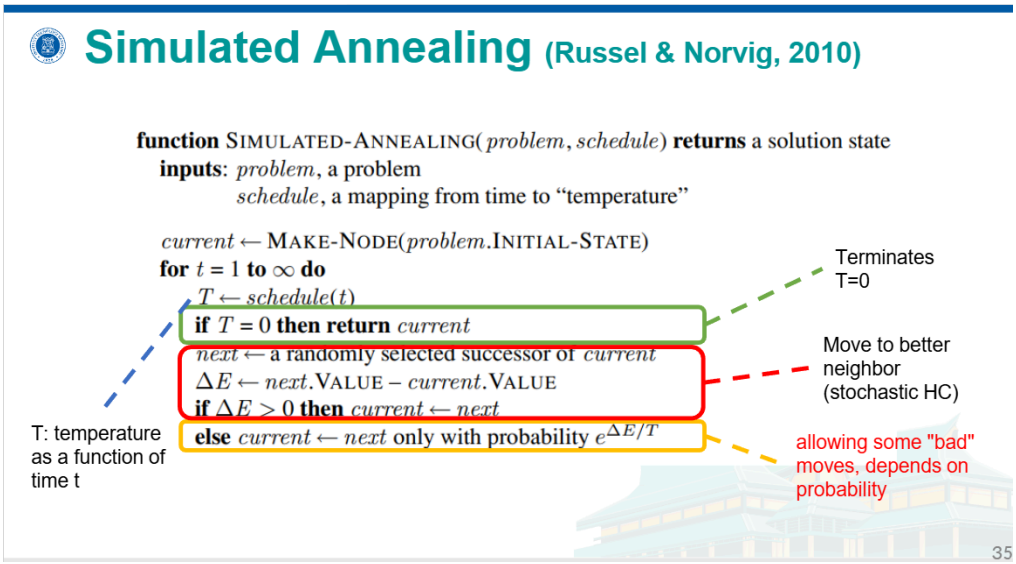
```
run in runs),
        restart_boundaries=tuple(boundaries),
    )
```

Proses utama berjalan pada fungsi random_restart_hill_climbing(). Algoritma ini menjalankan Steepest Ascent secara berulang dari state awal yang berbeda. Parameter max_restarts menentukan jumlah restart maksimum setelah pencarian pertama, sehingga jumlah pencarian maksimum adalah max_restarts + 1. Alurnya adalah sebagai berikut:

1. Buat state awal menggunakan _prepare_initial_state() dan inisialisasi variabel best_state untuk menyimpan solusi terbaik dari seluruh pencarian.
2. Jalankan Steepest Ascent menggunakan _steepest_run() hingga pencarian mencapai local optimum. Simpan hasil run beserta nilai objective akhirnya.
3. Bandingkan nilai objective dari hasil run dengan best_value. Apabila hasil run memiliki nilai lebih tinggi, perbarui best_state dan best_value dengan solusi tersebut.
4. Periksa apakah best_value telah mencapai optimum_upper_bound, yaitu jumlah nilai seluruh paket. Apabila tercapai, pencarian langsung dihentikan karena nilai objective tidak mungkin melampaui batas tersebut.
5. Jika batas tersebut belum tercapai dan masih tersedia kesempatan restart, bangkitkan state awal baru secara acak menggunakan random_initial_state(), kemudian ulangi langkah 2–4.
6. Setelah seluruh run selesai, gabungkan riwayat objective setiap run ke dalam objective_history. Simpan pula restart_boundaries untuk menandai titik awal setiap run pada grafik.
7. Kembalikan HillClimbingResult yang memuat solusi terbaik dari seluruh run, nilai objective akhir, jumlah restart, jumlah iterasi setiap run, total iterasi, durasi pencarian, riwayat objective, serta alasan terminasi.

2.3. Simulated Annealing

Implementasi Simulated Annealing pada tugas ini didasarkan pada pseudocode di slide Local Search IF3170 yang seperti ini:



Kodenya sebagai berikut:

```
while temperature > min_temperature:
        if iterations >= max_iterations:
            termination_reason = "max_iterations"
            break
        iterations += 1

        neighbor = generate_valid_neighbor(problem, current,
generator)
        probability = 1.0

        if neighbor is None:
            no_neighbor_count += 1
        else:
            neighbor_value = objective(problem, neighbor)
            delta = neighbor_value - current_value

            if delta >= 0:
                accepted = True
            else:
                probability = math.exp(delta / temperature)
                accepted = generator.random() < probability
                if not accepted:
                    stuck_count += 1

            if accepted:
```

```
                current, current_value = neighbor,
neighbor_value
                if current_value > best_value:
                    best, best_value = deepcopy(current),
current_value

        acceptance_history.append(probability)
        history.append(current_value)
        temperature *= cooling_rate
```

Terdapat sedikit modifikasi dari algoritma Simulated_Annealing dengan menggunakan T< min_temperature untuk menghentikan iterasi dari algoritma.

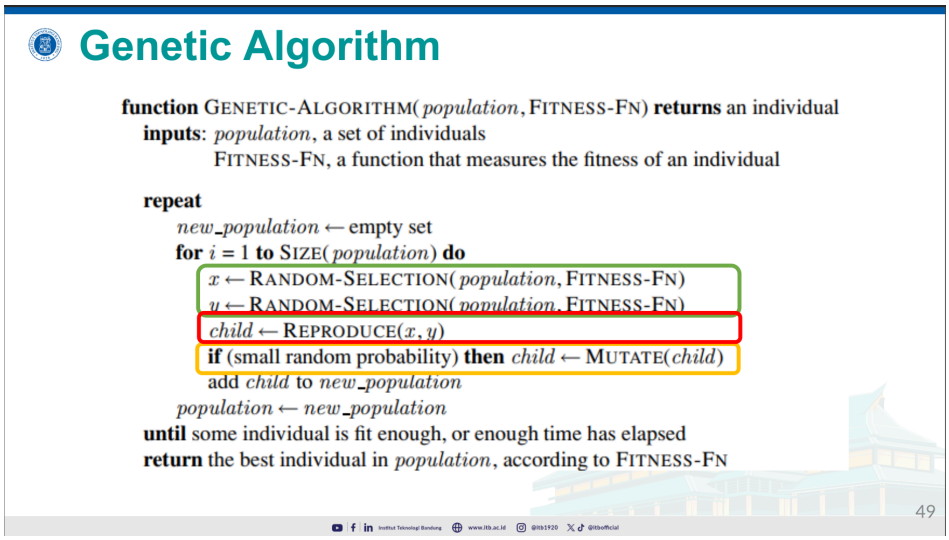
Proses utama berjalan pada fungsi simulated_annealing(). Parameter initial_temparature digunakan sebagai temperatur awal saat iterasi berjalan, min_temperature digunakan sebagai temperatur minimum untuk menghentikan iterasi, cooling_rate digunakan sebagai ratio perkurangan temperatur pada tiap iterasi. max_iterations merupakan jumlah iterasi maksimum yang bisa dilakukan oleh algoritma. Alurnya sebagai berikut:

1. Buat Sebuah State awal yang random, state menyimpan posisi dan orientasi seluruh paket, termasuk paket di luar truk. Susunan yang dibentuk harus memenuhi constraint.
2. Generate sebuah state bertetangga yang random, lalu dihitung berapa value dari state tersebut
3. Bila value dari state lebih tinggi akan berpindah state menjadi state tersebut bila lebih rendah akan melakukan hal berikut.
    a. Hitung probabilitas dari State tersebut dengan fungsi $e^{\Delta E/T}$ dengan ΔE merupakan value state tetangga dikurang value state sekarang, dan T merupakan temperature.
    b. Bila hasil $e^{\Delta E/T}$ lebih tinggi dari random(0,1) maka state tersebut akan diambil.
    c. Bila hasil $e^{\Delta E/T}$ lebih rendah dari random(0,1) maka state akan diabaikan dan tidak berpindah.
4. Temperatur akan berkurang berdasarkan dengan cooling_rate

5. Dilakukan berulang-ulang hingga temperatur lebih kecil dari min_temperature atau iterasi sudah habis

2.4. Genetic Algorithm

Implementasi Genetic Algorithm pada tugas ini didasarkan pada pseudocode di slide Local Search IF3170 yang seperti ini:



Kodenya sebagai berikut:

```
for _ in range(max_iterations):
        highest = max(fitness)
        weights = [value / highest for value in fitness] if highest > 0
else None
        highest_index = max(range(population_size),
key=fitness.__getitem__)
        new_population = [deepcopy(population[highest_index])]
        for _ in range(population_size - 1):
            first, second = generator.choices(population,
weights=weights, k=2)
            child = _crossover(first, second, package_ids, generator)
            if len(package_ids) >= 2 and not is_valid(problem, child):
                child = deepcopy(first)

            if generator.random() < mutation_rate:
                mutated = generate_valid_neighbor(problem, child,
generator)
                if mutated is not None:
                    child = mutated
            new_population.append(child)
```

```
        population = new_population
        fitness = [objective(problem, state) for state in population]
        maxima.append(max(fitness))
        means.append(sum(value / population_size for value in fitness))
```

Proses utama berjalan pada fungsi genetic_algorithm(). Alurnya adalah sebagai berikut:

1. Buat N state random sebagai populasi awal. Setiap state menyimpan posisi dan orientasi seluruh paket, termasuk paket di luar truk. Susunan yang dibentuk harus memenuhi constraint.

2. Pilih dua parent secara random berbobot fitness. Individu dengan fitness lebih tinggi memiliki peluang lebih besar untuk terpilih. Misalnya terdapat empat individu:

Individu	Fitness	Peluang	Rentang
S1	20	20%	0-20
S2	25	25%	20-45
S3	35	35%	45-80
S4	20	20%	80-100

   Jika undian pertama menghasilkan 40%, parent pertama adalah S2. Jika undian kedua menghasilkan 70%, parent kedua adalah S3. Parent tetap tersedia untuk undian berikutnya, sehingga individu yang sama boleh terpilih berulang.

3. Gabungkan kedua parent melalui crossover menjadi satu child. Program memilih titik potong random pada urutan paket. Child mengambil penempatan paket sebelum titik potong dari parent pertama dan sisanya dari parent kedua. Misalnya:

   - Parent 1: A2, B2 | C2, D2
   - Parent 2: A3, B3 | C3, D3
   - Child: A2, B2 | C3, D3

Notasi A2 berarti posisi dan orientasi paket A pada S2. Karena penggabungan dapat menyebabkan collision atau pelanggaran constraint lain, child diperiksa menggunakan is_valid(). Jika invalid, child diganti salinan parent pertama.

4. Lakukan mutasi dengan peluang kecil. Child dapat mengalami pemindahan, rotasi, atau pertukaran paket. Hasil mutasi diterima jika valid. Jika tidak ditemukan perubahan valid, child sebelumnya dipertahankan.

5. Ulangi langkah 2-4 sampai terkumpul N child. Seluruh parent masih dipilih dari populasi lama. Setelah jumlah child mencukupi, seluruh populasi lama diganti: [S1, S2, S3, S4] -> [child 1, child 2, child 3, child 4]

6. Ulangi proses sampai batas generasi. Fitness populasi baru dihitung kembali untuk menentukan peluang seleksi berikutnya. Program juga mencatat fitness maksimum dan rata-rata setiap generasi.

7. Ambil individu terbaik dari populasi terakhir. Hasil pencarian beserta state awal/akhir, riwayat fitness, konfigurasi, dan durasi dikembalikan melalui GeneticAlgorithmResult.

Penyesuaian utama terhadap pseudocode kuliah adalah penggunaan batas jumlah generasi dan pemeriksaan validitas children untuk memenuhi constraint penyusunan paket. Jika semua fitness bernilai nol, parent dipilih dengan peluang merata. Seluruh populasi diganti sehingga individu terbaik dari generasi sebelumnya tidak otomatis dipertahankan. State yang memiliki hasil terbaik saat ini selalu disimpan dan digunakan kembali di generasi berikutnya.

2.5. Eksperimen dan Analisis

2.5.1. Hill Climbing

Eksperimen Hill-Climbing dilakukan kepada keempat algoritma Hill-Climbing, yaitu steepest ascent, steepest ascent with sideways move, random restart hill climbing, dan stochastic hill climbing. Dilakukan pada tc1, tc2, dan tc3.

Hasil Steepest Ascent Hill-Climbing dari tc1
STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (0, 0, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: di luar truk | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

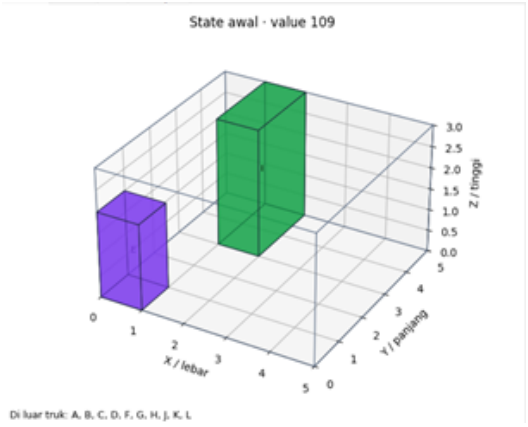
H: di luar truk | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1

I: (1, 3, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile



STATE AKHIR

A: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: (3, 1, 2) | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (0, 0, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: (2, 1, 1) | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

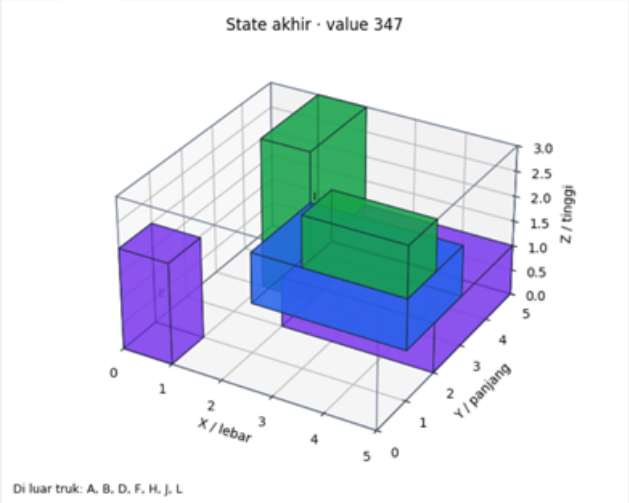
H: di luar truk | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1

I: (1, 3, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: (2, 2, 0) | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile False | ETA 2



Algoritma: Steepest Ascent Hill-Climbing

Seed: 42

Objective: Total value

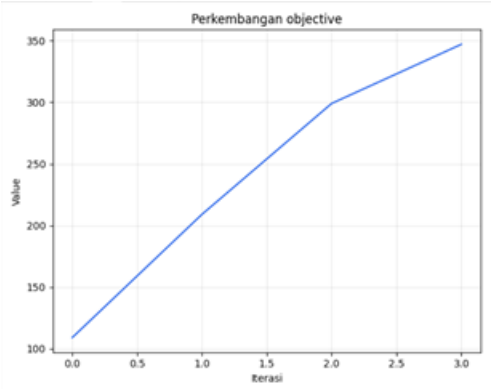
Skor objective awal → akhir: 109 → 347

Paket di truk: 5/12

Berat akhir: 30/30

Iterasi: 3

Durasi: 0.1433 detik



Hasil Steepest Ascent Hill-Climbing dari tc2

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4

F: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2

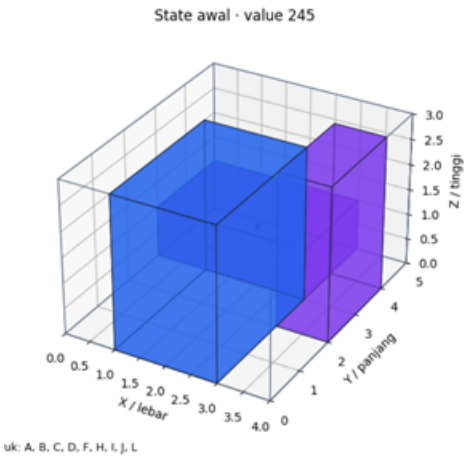
H: di luar truk | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1

J: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: (0, 3, 0) | orientasi XYZ | ukuran (3, 2, 1) | value 55 | berat 5 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile



STATE AKHIR

A: (0, 3, 1) | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4

F: (3, 4, 0) | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2

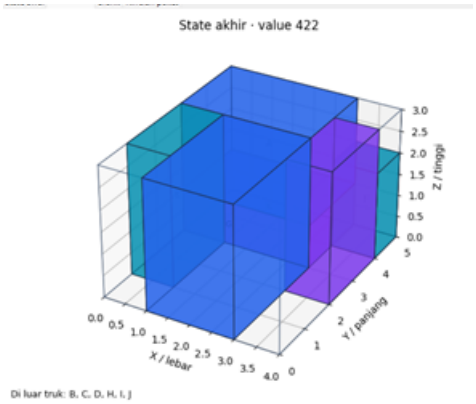
H: di luar truk | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1

J: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: (0, 3, 0) | orientasi XYZ | ukuran (3, 2, 1) | value 55 | berat 5 | fragile False | ETA 4

L: (0, 1, 0) | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile False | ETA 1



Algoritma: Steepest Ascent Hill-Climbing

Seed: 42

Objective: Total value

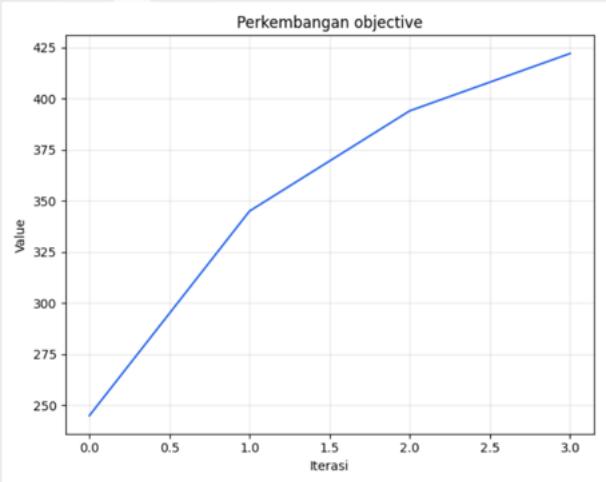
Skor objective awal → akhir: 245 → 422

Paket di truk: 6/12

Berat akhir: 36/100

Iterasi: 3

Durasi: 0.0862 detik



Hasil Steepest Ascent Hill-Climbing dari tc3

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: di luar truk | orientasi XYZ | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

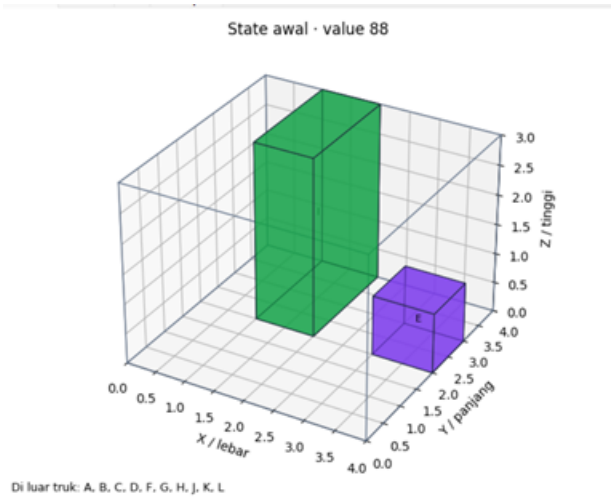
H: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (1, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 1, 2) | value 62 | berat 6 | fragile False | ETA 2

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



STATE AKHIR

A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2

B: (0, 0, 2) | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: di luar truk | orientasi XYZ | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1

D: (2, 3, 0) | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: (2, 2, 2) | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: (0, 0, 0) | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

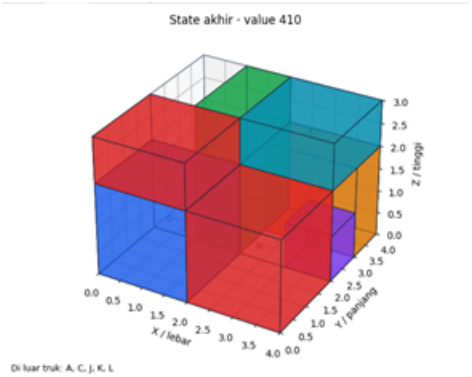
H: (2, 0, 0) | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (1, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 1, 2) | value 62 | berat 6 | fragile False | ETA 2

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



Algoritma: Steepest Ascent Hill-Climbing

Seed: 42

Objective: Total value

Skor objective awal → akhir: 88 → 410

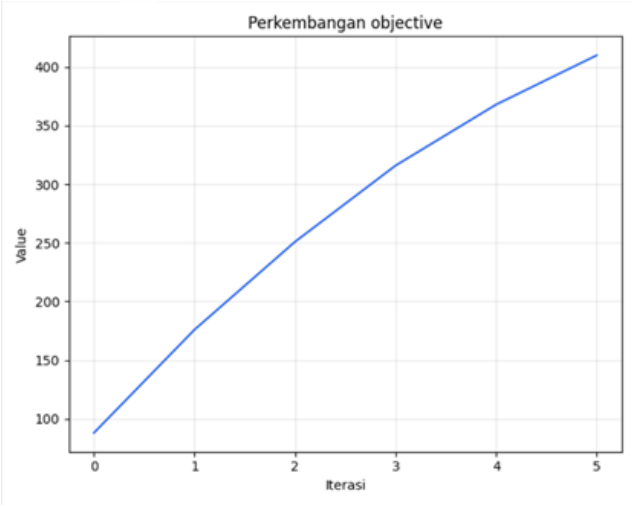
Paket di truk: 7/12

Berat akhir: 35/80

Iterasi: 5

Durasi: 0.1289 detik

Berhenti karena: local_optimum



Hasil Hill-Climbing with Sideways Move dari tc1

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (0, 0, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: di luar truk | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

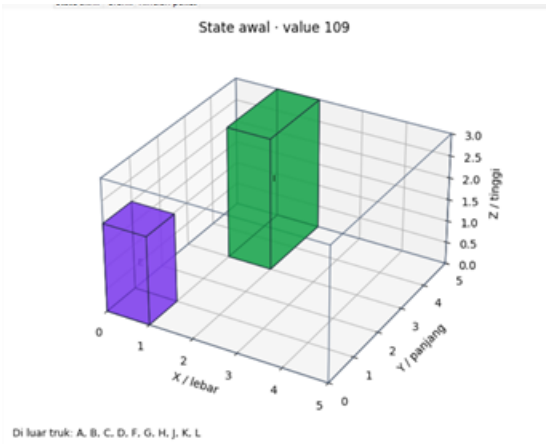
H: di luar truk | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1

I: (1, 3, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile False | ETA 2



STATE AKHIR

A: (1, 2, 0) | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: (0, 1, 0) | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi ZYX | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (3, 0, 0) | orientasi ZXY | ukuran (2, 1, 1) | value 35 | berat 2 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: (0, 2, 1) | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

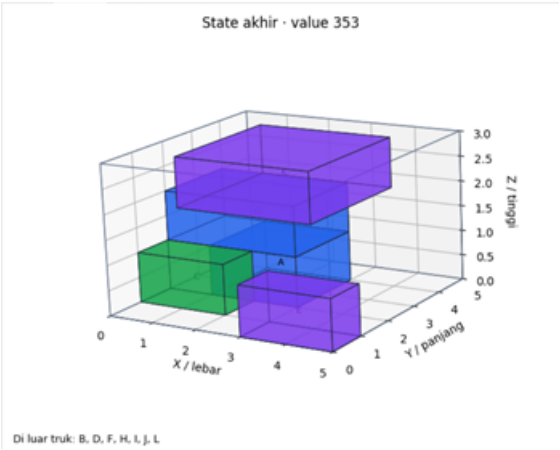
H: di luar truk | orientasi ZYX | ukuran (2, 3, 1) | value 78 | berat 7 | fragile False | ETA 1

I: di luar truk | orientasi YXZ | ukuran (3, 2, 1) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi YXZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: (1, 1, 2) | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile



Algoritma: Hill-Climbing with Sideways Move

Seed: 42

Objective: Total value

Skor objective awal → akhir: 109 → 353

Paket di truk: 5/12

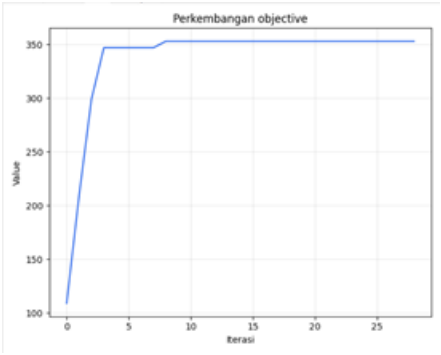
Berat akhir: 30/30

Iterasi: 28

Durasi: 1.1616 detik

Berhenti karena: max_sideways_moves

Sideways move: 24



Hasil Hill-Climbing with Sideways Move dari tc2

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4

F: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2

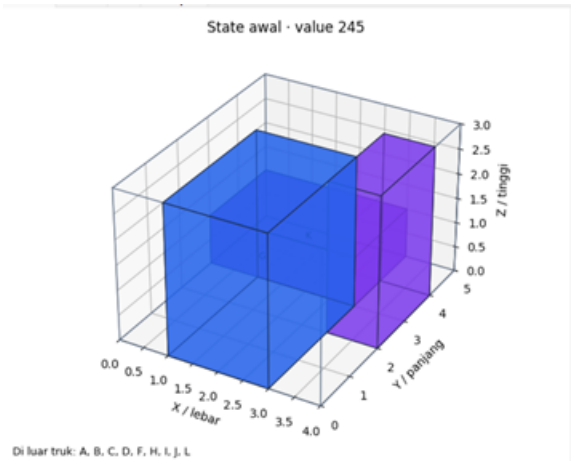
H: di luar truk | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1

J: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: (0, 3, 0) | orientasi XYZ | ukuran (3, 2, 1) | value 55 | berat 5 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile False | ETA 1



STATE AKHIR

A: (0, 3, 1) | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: di luar truk | orientasi ZXY | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: (0, 3, 0) | orientasi YXZ | ukuran (3, 2, 1) | value 70 | berat 6 | fragile False | ETA 1

D: (3, 1, 0) | orientasi ZXY | ukuran (1, 2, 2) | value 55 | berat 5 | fragile True | ETA 2

E: (3, 3, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4

F: (3, 0, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2

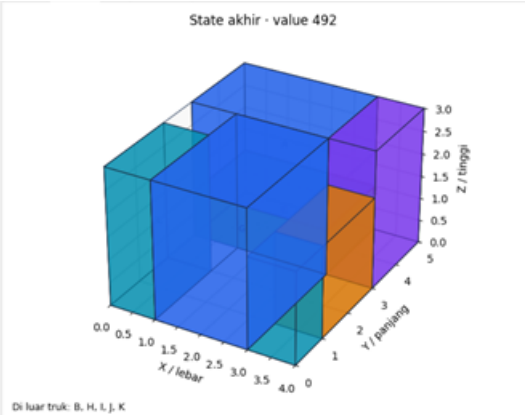
H: di luar truk | orientasi ZXY | ukuran (1, 2, 4) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi ZXY | ukuran (2, 1, 4) | value 68 | berat 6 | fragile False | ETA 1

J: di luar truk | orientasi YXZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: di luar truk | orientasi YZX | ukuran (2, 1, 3) | value 55 | berat 5 | fragile False | ETA 4

L: (0, 0, 0) | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile False | ETA 1



Algoritma: Hill-Climbing with Sideways Move

Seed: 42

Objective: Total value

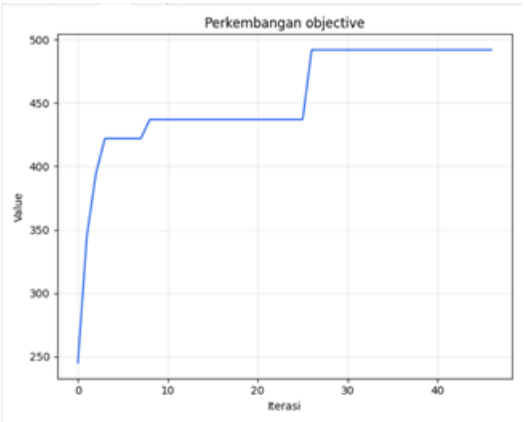
Skor objective awal → akhir: 245 → 492

Paket di truk: 7/12

Berat akhir: 42/100

Iterasi: 46

Durasi: 1.1166 detik



Hasil Hill-Climbing with Sideways Move dari tc3

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: di luar truk | orientasi XYZ | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

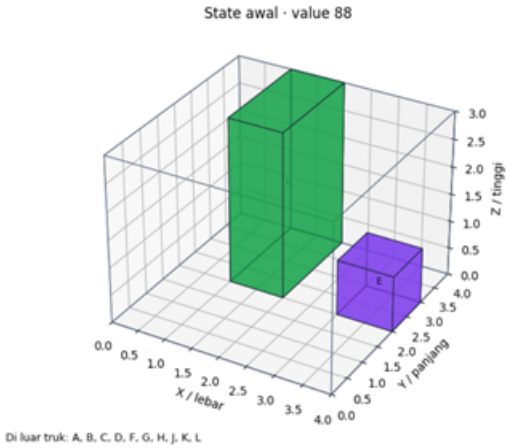
H: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (1, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 1, 2) | value 62 | berat 6 | fragile False | ETA 2

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



STATE AKHIR

A: di luar truk | orientasi YZX | ukuran (3, 1, 3) | value 70 | berat 6 | fragile False | ETA 2

B: (2, 0, 2) | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: di luar truk | orientasi YXZ | ukuran (3, 1, 1) | value 50 | berat 4 | fragile False | ETA 1

D: (2, 2, 0) | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2

E: (0, 3, 0) | orientasi YXZ | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: (2, 2, 2) | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: (2, 0, 0) | orientasi YZX | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

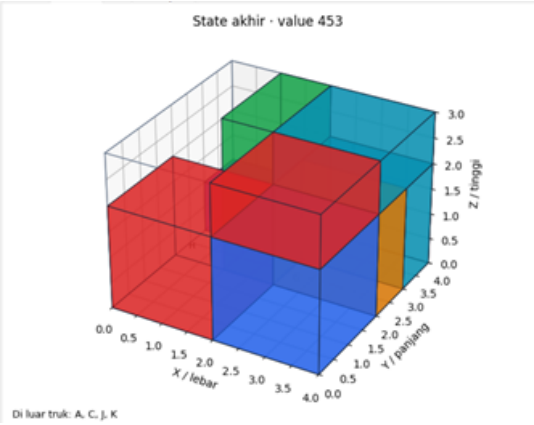
H: (0, 0, 0) | orientasi YZX | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (1, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: di luar truk | orientasi XZY | ukuran (3, 2, 1) | value 62 | berat 6 | fragile False | ETA 2

L: (2, 3, 0) | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



Algoritma: Hill-Climbing with Sideways Move

Seed: 42

Objective: Total value

Skor objective awal → akhir: 88 → 453

Paket di truk: 8/12

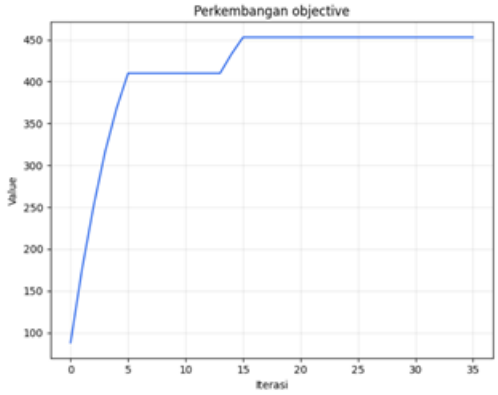
Berat akhir: 39/80

Iterasi: 35

Durasi: 0.9358 detik

Berhenti karena: max_sideways_moves

Sideways move: 28



Hasil Random Restart Hill-Climbing dari tc1

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (0, 0, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: di luar truk | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

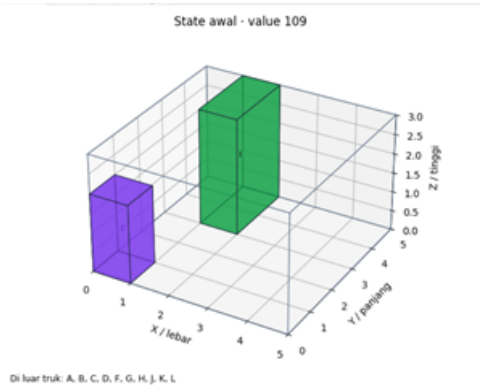
H: di luar truk | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1

I: (1, 3, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile False | ETA 2



STATE AKHIR

A: (2, 0, 0) | orientasi ZYX | ukuran (1, 2, 2) | value 80 | berat 6 | fragile False | ETA 2

B: (0, 3, 0) | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: (4, 1, 0) | orientasi ZYX | ukuran (1, 1, 2) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (0, 2, 0) | orientasi XYZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1

F: (0, 0, 0) | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: di luar truk | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

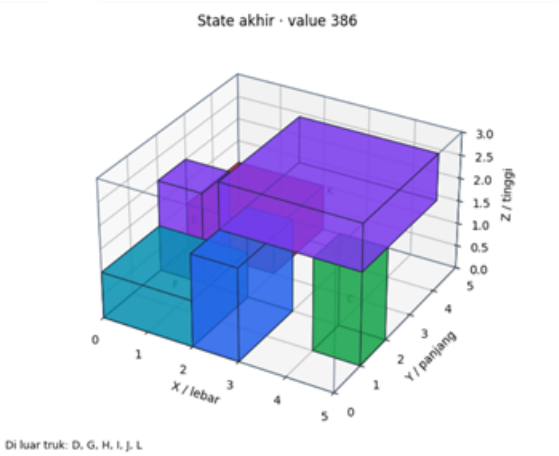
H: di luar truk | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1

I: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: (2, 1, 2) | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile False | ETA 2



Algoritma: Random Restart Hill-Climbing

Seed: 42

Objective: Total value

Skor objective awal → akhir: 109 → 386

Paket di truk: 6/12

Berat akhir: 30/30

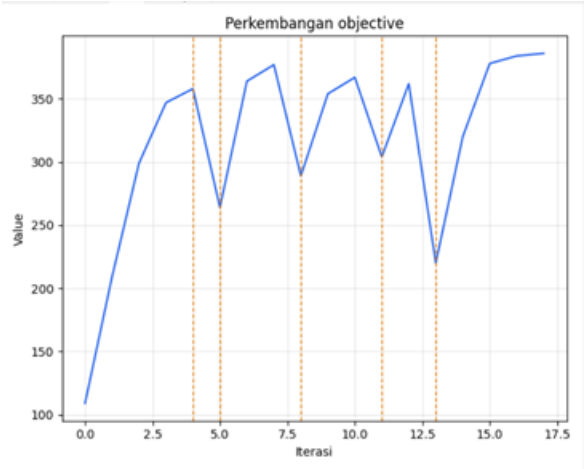
Iterasi: 12

Durasi: 0.7129 detik

Berhenti karena: max_restarts

Restart: 5

Iterasi tiap run: [3, 0, 2, 2, 1, 4]



Hasil Random Restart Hill-Climbing dari tc2

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4

F: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2

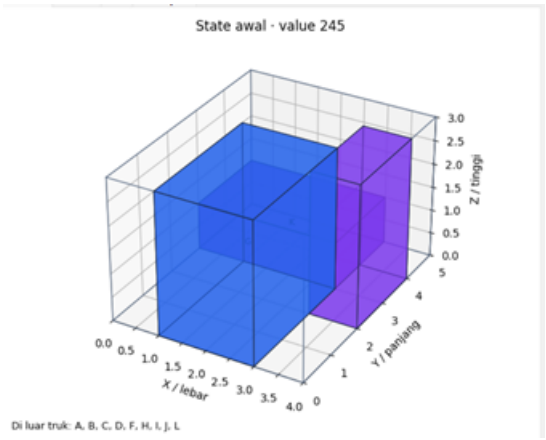
H: di luar truk | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1

J: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: (0, 3, 0) | orientasi XYZ | ukuran (3, 2, 1) | value 55 | berat 5 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile



STATE AKHIR

A: di luar truk | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: (2, 0, 0) | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: (2, 2, 0) | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1

D: (2, 0, 2) | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2

E: (0, 2, 1) | orientasi XYZ | ukuran (1, 3, 2) | value 65 | berat 5 | fragile False | ETA 4

F: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 2, 1) | orientasi XYZ | ukuran (3, 3, 2) | value 125 | berat 12 | fragile False | ETA 2

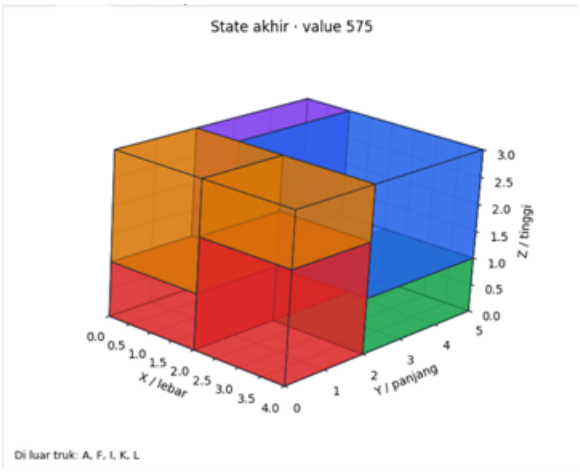
H: (0, 0, 0) | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1

J: (0, 0, 1) | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: di luar truk | orientasi YZX | ukuran (2, 1, 3) | value 55 | berat 5 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile



Algoritma: Random Restart Hill-Climbing

Seed: 42

Objective: Total value

Skor objective awal → akhir: 245 → 575

Paket di truk: 7/12

Berat akhir: 52/100

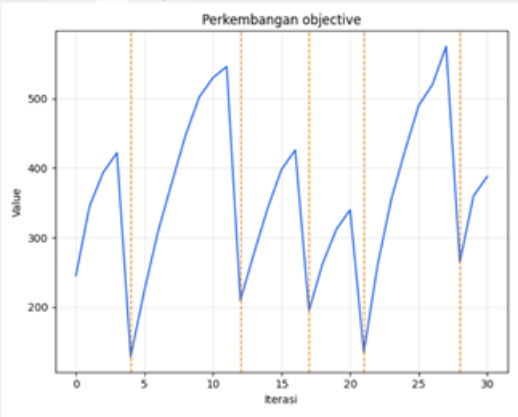
Iterasi: 25

Durasi: 0.6755 detik

Berhenti karena: max_restarts

Restart: 5

Iterasi tiap run: [3, 7, 4, 3, 6, 2]



Hasil Random Restart Hill-Climbing dari tc3

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: di luar truk | orientasi XYZ | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

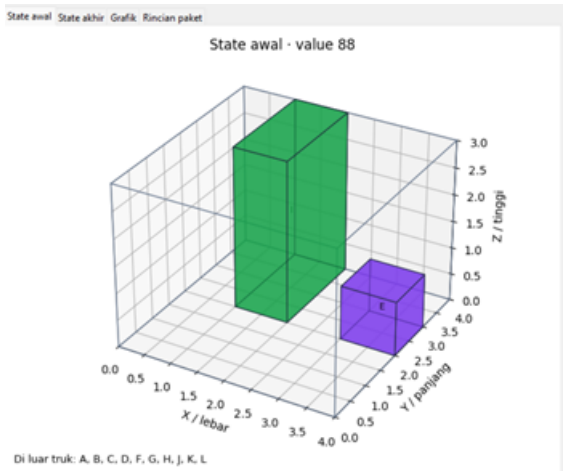
H: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (1, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 1, 2) | value 62 | berat 6 | fragile False | ETA 2

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



STATE AKHIR

A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2

B: (2, 2, 2) | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: (1, 3, 0) | orientasi XZY | ukuran (1, 1, 3) | value 50 | berat 4 | fragile False | ETA 1

D: (1, 0, 0) | orientasi XZY | ukuran (2, 2, 1) | value 52 | berat 4 | fragile False | ETA 2

E: (2, 3, 0) | orientasi XYZ | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

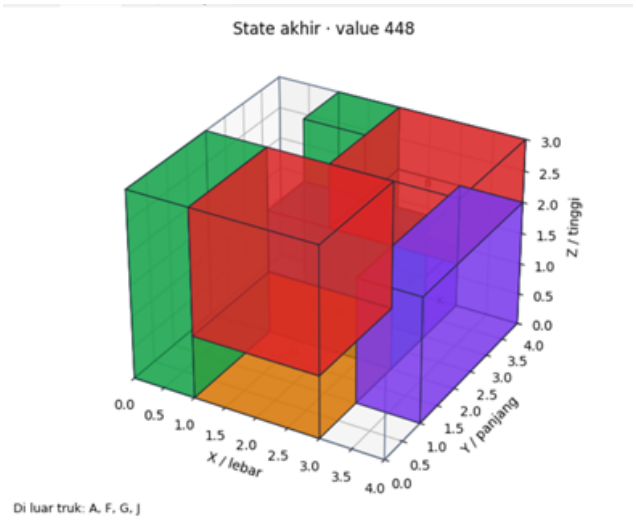
H: (1, 0, 1) | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (0, 0, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: (3, 1, 0) | orientasi YXZ | ukuran (1, 3, 2) | value 62 | berat 6 | fragile False | ETA 2

L: (1, 2, 0) | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



Algoritma: Random Restart Hill-Climbing

Seed: 42

Objective: Total value

Skor objective awal → akhir: 88 → 448

Paket di truk: 8/12

Berat akhir: 39/80

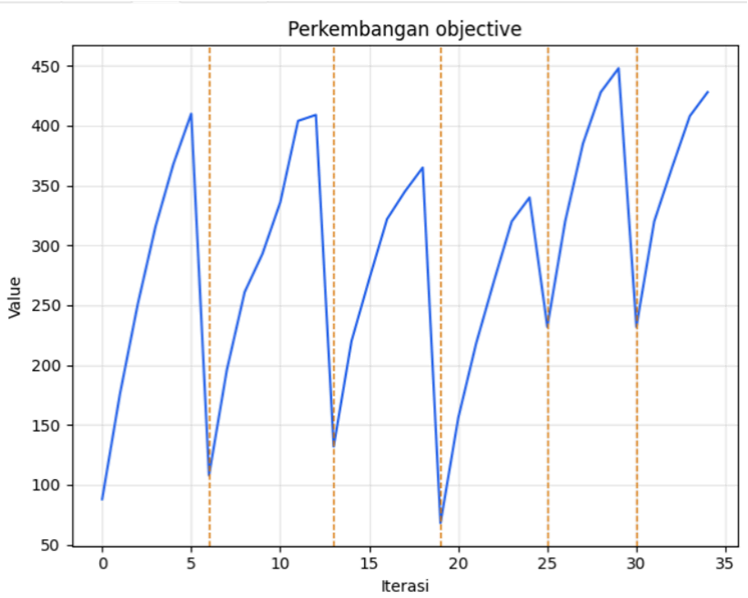
Iterasi: 29

Durasi: 0.7872 detik

Berhenti karena: max_restarts

Restart: 5

Iterasi tiap run: [5, 6, 5, 5, 4, 4]



Hasil Stochastic Hill-Climbing dari tc1

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (0, 0, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: di luar truk | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

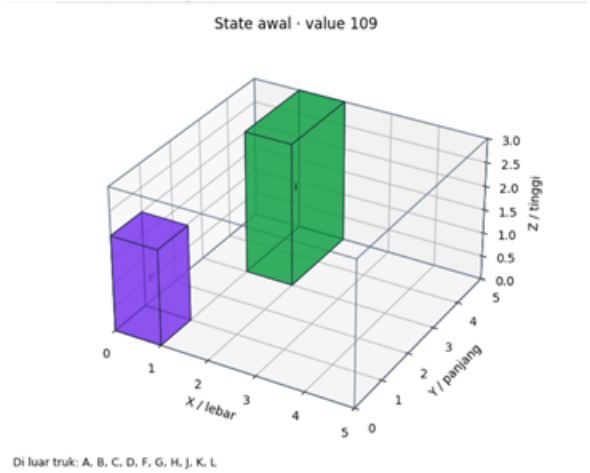
H: di luar truk | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1

I: (1, 3, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile Fals



STATE AKHIR

A: (1, 3, 0) | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2

E: (0, 3, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1

F: (2, 1, 0) | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4

G: di luar truk | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2

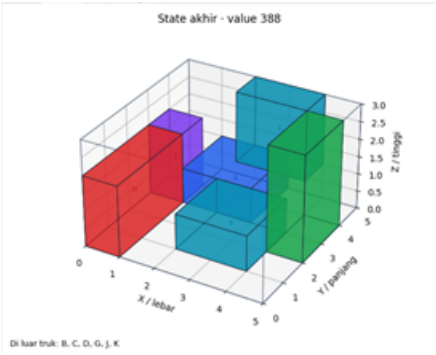
H: (0, 0, 0) | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1

I: (4, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 74 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4

L: (2, 4, 1) | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile False | ETA 2



Algoritma: Stochastic Hill-Climbing

Seed: 42

Objective: Total value

Skor objective awal → akhir: 109 → 388

Paket di truk: 6/12

Berat akhir: 30/30

Iterasi: 100

Durasi: 0.0152 detik
Berhenti karena: max_iterations

Hasil Stochastic Hill-Climbing dari tc2

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4

F: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2

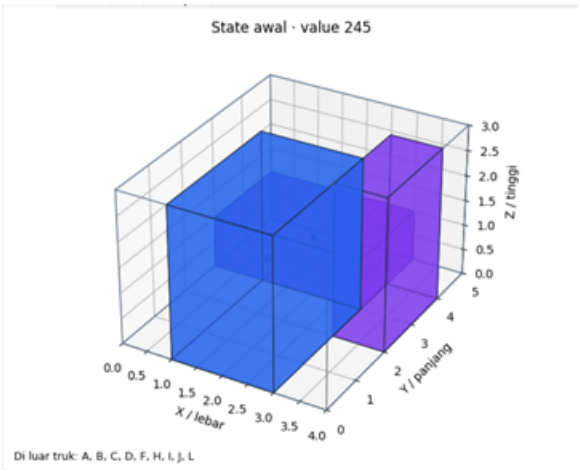
H: di luar truk | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1

J: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: (0, 3, 0) | orientasi XYZ | ukuran (3, 2, 1) | value 55 | berat 5 | fragile False | ETA 4

L: di luar truk | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile



STATE AKHIR

A: di luar truk | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3

C: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4

F: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1

G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2

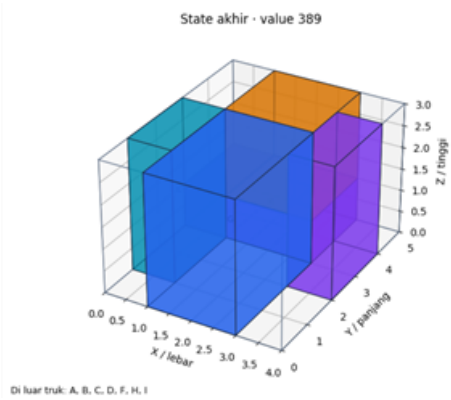
H: di luar truk | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3

I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1

J: (1, 3, 1) | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2

K: (0, 3, 0) | orientasi XYZ | ukuran (3, 2, 1) | value 55 | berat 5 | fragile False | ETA 4

L: (0, 1, 0) | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile False | ETA 1



Algoritma: Stochastic Hill-Climbing

Seed: 42

Objective: Total value

Skor objective awal → akhir: 245 → 389

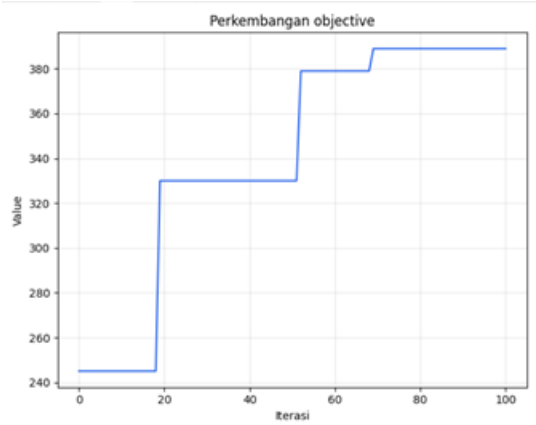
Paket di truk: 5/12

Berat akhir: 35/100

Iterasi: 100

Durasi: 0.0207 detik

Berhenti karena: max_iterations



Hasil Stochastic Hill-Climbing dari tc3

STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2

B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: di luar truk | orientasi XYZ | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2

E: (3, 2, 0) | orientasi XZY | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

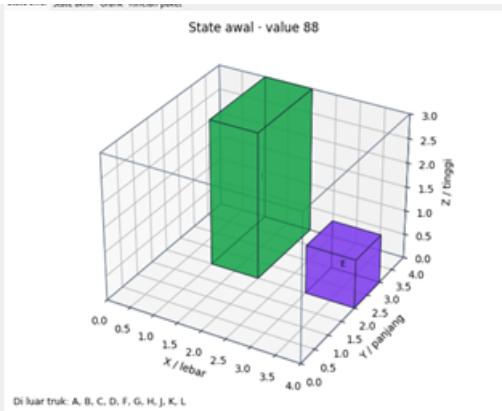
H: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (1, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: di luar truk | orientasi XYZ | ukuran (3, 1, 2) | value 62 | berat 6 | fragile False | ETA 2

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



STATE AKHIR

A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2

B: (1, 0, 2) | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3

C: di luar truk | orientasi XYZ | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1

D: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2

E: (0, 2, 0) | orientasi XZY | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1

F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4

G: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3

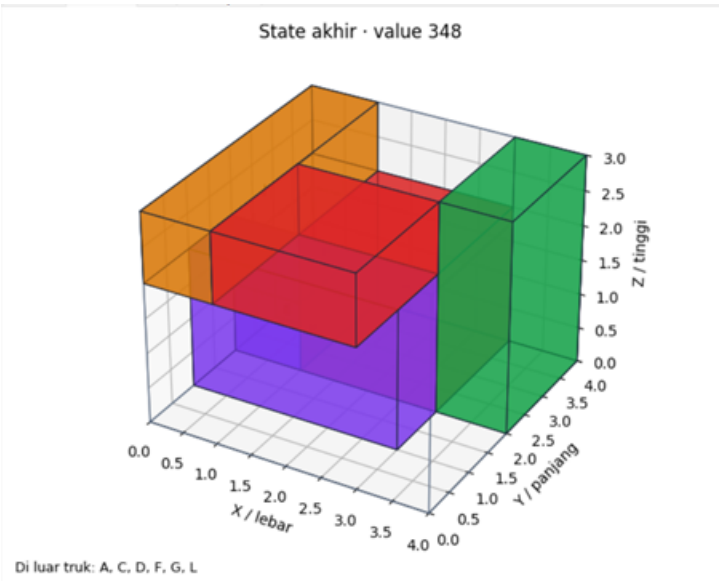
H: (1, 2, 0) | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2

I: (3, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3

J: (0, 0, 2) | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1

K: (0, 1, 0) | orientasi XYZ | ukuran (3, 1, 2) | value 62 | berat 6 | fragile False | ETA 2

L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



Algoritma: Stochastic Hill-Climbing

Seed: 42

Objective: Total value

Skor objective awal → akhir: 88 → 348

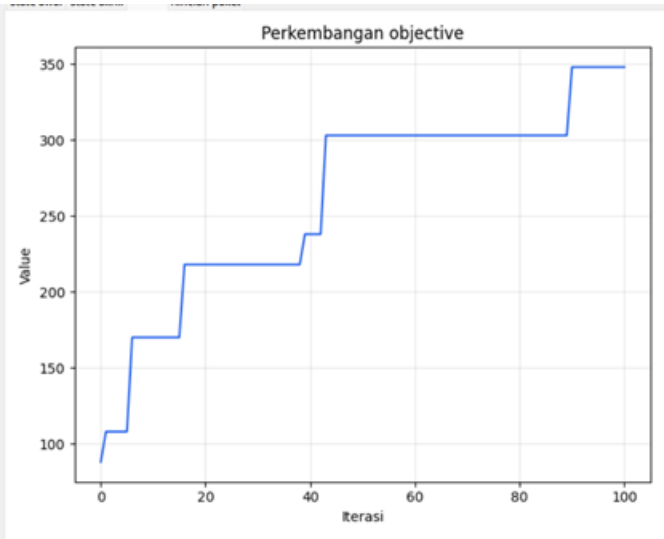
Paket di truk: 6/12

Berat akhir: 31/80

Iterasi: 100

Durasi: 0.0191 detik

Berhenti karena: max_iterations



Berdasarkan hasil dari keempat algoritma hill climbing, didapatkan bahwa steepest ascent memiliki waktu tercepat karena algoritma tersebut berhenti ketika telah mencapai satu local optimum, tetapi algoritma tersebut menyebabkan hasil sering tidak mendekati global optimum sebab tersangkut di local optimum

Algoritma steepest ascent dengan sideways memberikan hasil yang sedikit lebih tinggi valuenya dibanding dengan steepest ascent karena memperbolehkan eksplorasi tetangga yang valuenya sama dengannya sehingga tidak stuck di sebuah flat dan bisa bergerak keluar, tetapi masih terdapat permasalahan dan menghabiskan durasi jika algoritma sering bertemu flat lainnya yang berdekatan dan akan melakukan pergerakan pada flat yang sama dan tidak bertambah.

Algoritma Random Restart memberikan hasil yang paling tinggi dibanding algoritma lainnya karena melakukan steepest ascent yang diulang dengan state awal yang random sehingga dapat mengeksplorasi state lebih banyak. Terdapat kekurangan yaitu waktu iterasi yang lebih lambat karena harus melakukan steepest ascent berkali-kali jika jumlah restart sangat banyak.

Algoritma Stochastic memberikan hasil yang mendekati random restart tetapi tidak perlu menghabiskan waktu yang lebih banyak karena terbatasi dengan hanya memilih state yang lebih baik meskipun random. Stochastic masih akan tersangkut di local optima jika state random yang dibanding tetap lebih kecil. Stochastic masih menghasilkan hasil yang lebih dibanding dengan steepest ascent karena terdapat eksplorasi state tetangga yang lebih baik dan tidak harus yang terbaik dan memiliki waktu yang lebih cepat dibanding dengan random restart.

Berdasarkan eksperimen pada tiga test case, seluruh varian Hill Climbing berhasil meningkatkan nilai objective dari state awal menuju state akhir. Pada Steepest Ascent, peningkatan objective untuk tc1, tc2, dan tc3 masing-masing sebesar 238, 177, dan 322. Sideways Move menghasilkan peningkatan sebesar 244, 247, dan 365, sedangkan Random Restart menghasilkan peningkatan sebesar 277, 330, dan 360. Stochastic Hill Climbing menghasilkan peningkatan sebesar 279, 144, dan 260. Hasil tersebut menunjukkan bahwa besarnya peningkatan objective bergantung pada varian algoritma dan karakteristik konfigurasi awal setiap kasus.

Dari segi kualitas solusi, Random Restart memperoleh rata-rata objective akhir tertinggi sebesar 469,67, diikuti oleh Sideways Move sebesar 432,67, Steepest Ascent sebesar 393, dan Stochastic sebesar 375. Random Restart dapat memperoleh solusi yang lebih baik karena melakukan pencarian berulang dari beberapa state awal acak, sehingga memiliki peluang mengeksplorasi wilayah ruang solusi yang berbeda. Sideways Move

juga menghasilkan nilai lebih tinggi daripada Steepest Ascent pada ketiga kasus karena memperbolehkan perpindahan menuju tetangga dengan objective yang sama.

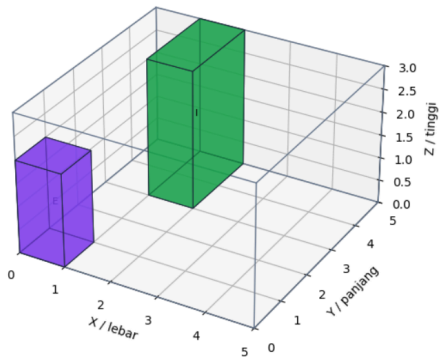
Berdasarkan grafik perkembangan objective, Steepest Ascent menunjukkan peningkatan pada setiap perpindahan hingga tidak ditemukan tetangga yang lebih baik. Sideways Move memperlihatkan periode objective konstan selama melakukan perpindahan pada plateau sebelum kembali meningkat atau mencapai batas sideways move. Stochastic menunjukkan peningkatan secara bertahap yang diselingi periode tanpa perubahan objective karena tetangga acak yang tidak lebih baik ditolak. Sementara itu, Random Restart memperlihatkan beberapa rangkaian peningkatan objective yang terpisah oleh restart. Pada setiap restart, pencarian dimulai dari state acak baru sehingga nilai objective dapat turun sebelum meningkat kembali.

2.5.2. Simulated Annealing

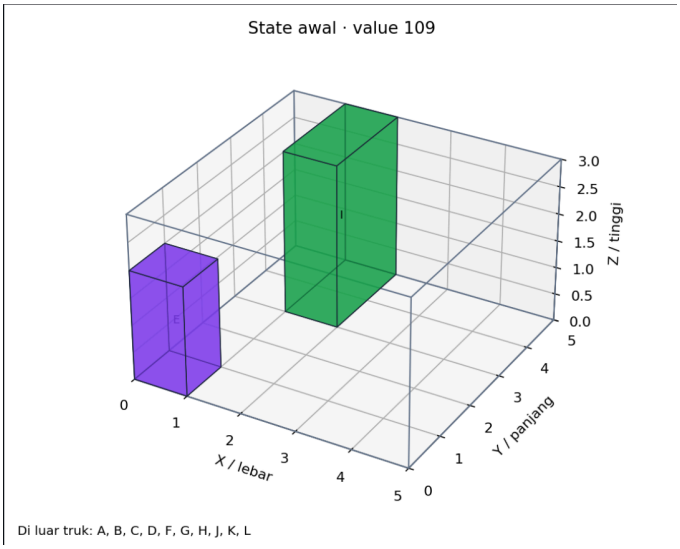
Eksperimen dilakukan dengan maksimum iterasi 1000, temperatur 10000, ration pendinginan 0.95 dan suhu minimum 0.01. Eksperimen ini dilakukan pada tiga test case yaitu tc1, tc2, tc3

Hasil Simulated Annealing dari tc1

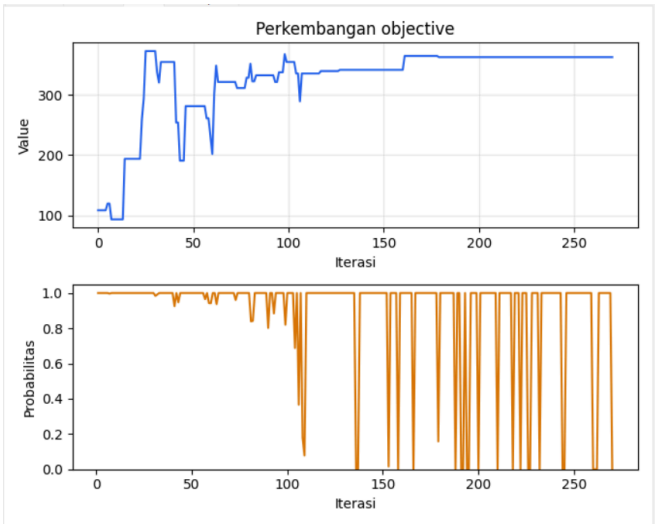
STATE AWAL
A: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 80 | berat 6 | fragile False | ETA 2
B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile False | ETA 3
C: di luar truk | orientasi XYZ | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1
D: di luar truk | orientasi XYZ | ukuran (1, 2, 1) | value 46 | berat 3 | fragile False | ETA 2
E: (0, 0, 0) | orientasi YXZ | ukuran (1, 1, 2) | value 35 | berat 2 | fragile True | ETA 1
F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 58 | berat 4 | fragile True | ETA 4
G: di luar truk | orientasi XYZ | ukuran (3, 2, 1) | value 90 | berat 9 | fragile False | ETA 2
H: di luar truk | orientasi XYZ | ukuran (1, 3, 2) | value 78 | berat 7 | fragile False | ETA 1
I: (1, 3, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 74 | berat 6 | fragile False | ETA 3
J: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 29 | berat 2 | fragile True | ETA 1
K: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4
L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 63 | berat 5 | fragile False | ETA 2



STATE AKHIR
A: di luar truk | orientasi XZY | ukuran (2, 1, 2) | value 80 | berat 6 | fragile False | ETA 2
B: (1, 4, 0) | orientasi YZX | ukuran (2, 1, 2) | value 65 | berat 5 | fragile False | ETA 3
C: (1, 3, 0) | orientasi XZY | ukuran (2, 1, 1) | value 48 | berat 3 | fragile False | ETA 1
D: (4, 2, 0) | orientasi XZY | ukuran (1, 1, 2) | value 46 | berat 3 | fragile False | ETA 2
E: (1, 1, 2) | orientasi YZX | ukuran (1, 2, 1) | value 35 | berat 2 | fragile True | ETA 1
F: di luar truk | orientasi XZY | ukuran (2, 1, 2) | value 58 | berat 4 | fragile True | ETA 4
G: di luar truk | orientasi XZY | ukuran (3, 1, 2) | value 90 | berat 9 | fragile False | ETA 2
H: (0, 0, 0) | orientasi YXZ | ukuran (3, 1, 2) | value 78 | berat 7 | fragile False | ETA 1
I: di luar truk | orientasi ZYX | ukuran (1, 3, 2) | value 74 | berat 6 | fragile False | ETA 3
J: di luar truk | orientasi YZX | ukuran (1, 2, 1) | value 29 | berat 2 | fragile True | ETA 1
K: (0, 1, 1) | orientasi XYZ | ukuran (3, 3, 1) | value 100 | berat 10 | fragile False | ETA 4
L: di luar truk | orientasi YXZ | ukuran (1, 2, 2) | value 63 | berat 5 | fragile False | ETA 2



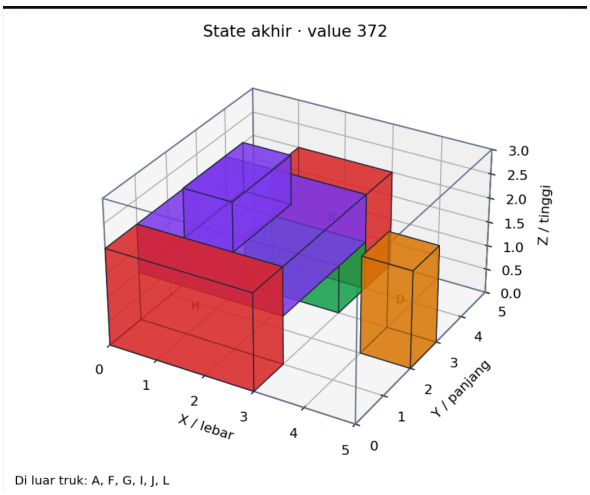
Objective: Total value
Skor objective awal → akhir: 109 → 372
Paket di truk: 6/12
Berat akhir: 30/30
Iterasi: 270
Durasi: 0.0508 detik
Berhenti karena: temperature_min
Stuck: 27



Hasil Simulated Annealing dari tc2

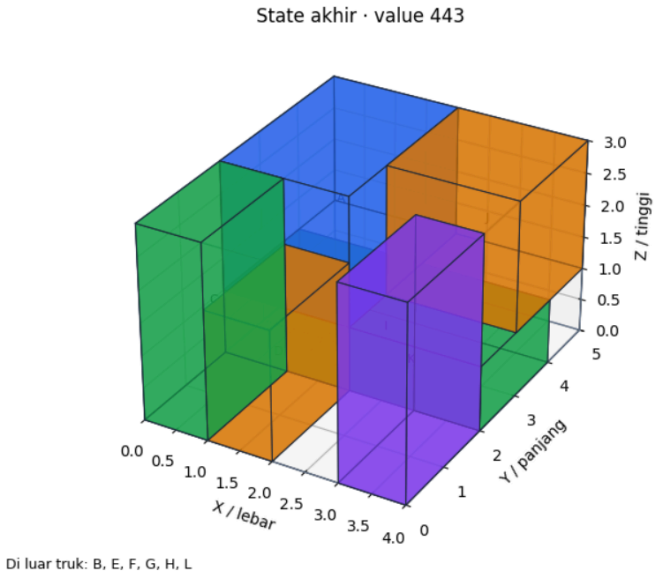
STATE AWAL

A: di luar truk | orientasi XYZ | ukuran (3, 2, 2) | value 100 | berat 8 | fragile False | ETA 2
B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3
C: di luar truk | orientasi XYZ | ukuran (2, 3, 1) | value 70 | berat 6 | fragile False | ETA 1
D: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 55 | berat 5 | fragile True | ETA 2
E: (3, 2, 0) | orientasi XZY | ukuran (1, 2, 3) | value 65 | berat 5 | fragile False | ETA 4
F: di luar truk | orientasi XYZ | ukuran (1, 1, 2) | value 28 | berat 2 | fragile True | ETA 1
G: (1, 0, 0) | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2
H: di luar truk | orientasi XYZ | ukuran (2, 4, 1) | value 80 | berat 8 | fragile False | ETA 3
I: di luar truk | orientasi XYZ | ukuran (1, 4, 2) | value 68 | berat 6 | fragile False | ETA 1
J: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2
K: (0, 3, 0) | orientasi XYZ | ukuran (3, 2, 1) | value 55 | berat 5 | fragile False | ETA 4
L: di luar truk | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile



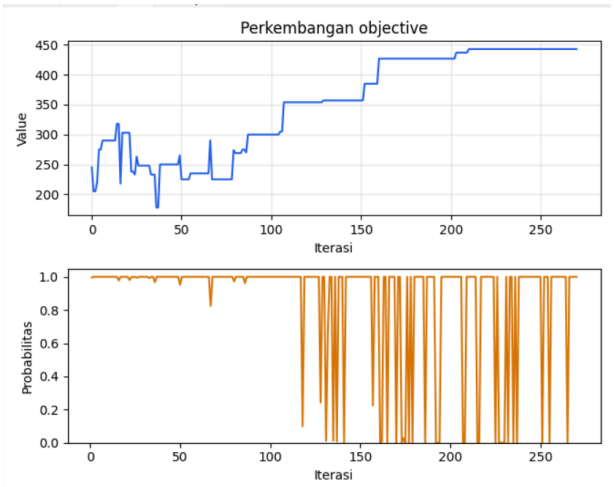
STATE AKHIR

A: (0, 2, 1) | orientasi ZXY | ukuran (2, 3, 2) | value 100 | berat 8 | fragile False | ETA 2
B: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 85 | berat 7 | fragile False | ETA 3
C: (0, 0, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 70 | berat 6 | fragile False | ETA 1
D: (1, 0, 0) | orientasi ZXY | ukuran (1, 2, 2) | value 55 | berat 5 | fragile True | ETA 2
E: di luar truk | orientasi ZYX | ukuran (2, 3, 1) | value 65 | berat 5 | fragile False | ETA 4
F: di luar truk | orientasi ZXY | ukuran (2, 1, 1) | value 28 | berat 2 | fragile True | ETA 1
G: di luar truk | orientasi ZXY | ukuran (2, 3, 3) | value 125 | berat 12 | fragile False | ETA 2
H: di luar truk | orientasi YZX | ukuran (4, 1, 2) | value 80 | berat 8 | fragile False | ETA 3
I: (0, 2, 0) | orientasi YZX | ukuran (4, 2, 1) | value 68 | berat 6 | fragile False | ETA 1
J: (2, 3, 1) | orientasi YXZ | ukuran (2, 2, 2) | value 95 | berat 9 | fragile True | ETA 2
K: (3, 0, 0) | orientasi ZYX | ukuran (1, 2, 3) | value 55 | berat 5 | fragile False | ETA 4
L: di luar truk | orientasi XYZ | ukuran (1, 2, 3) | value 49 | berat 4 | fragile False | ETA 1



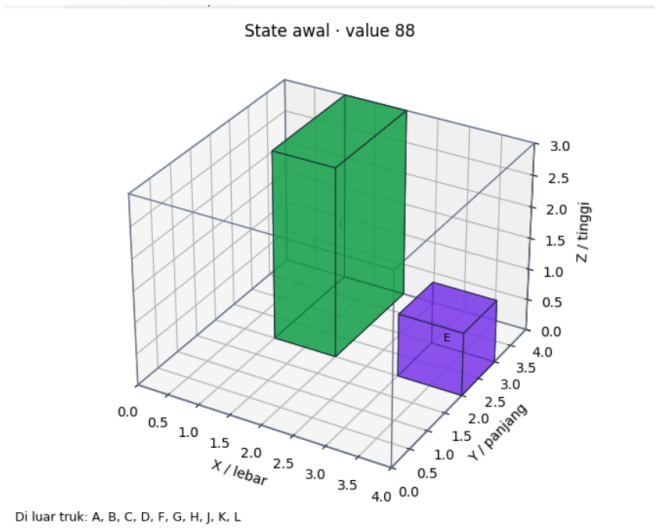
Algoritma: Simulated Annealing

Seed: 42
Objective: Total value
Skor objective awal → akhir: 245 → 443
Paket di truk: 6/12
Berat akhir: 39/100
Iterasi: 270
Durasi: 0.0614 detik



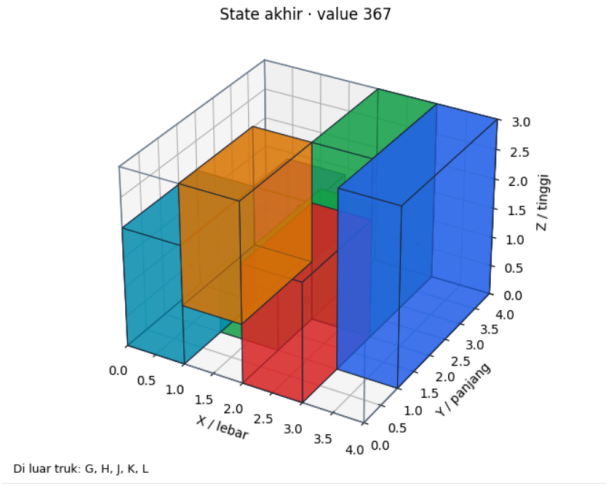
Hasil Simulated Annealing dari tc3

STATE AWAL
A: di luar truk | orientasi XYZ | ukuran (3, 3, 1) | value 70 | berat 6 | fragile False | ETA 2
B: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 65 | berat 5 | fragile True | ETA 3
C: di luar truk | orientasi XYZ | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1
D: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 52 | berat 4 | fragile False | ETA 2
E: (3, 2, 0) | orientasi XZY | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1
F: di luar truk | orientasi XYZ | ukuran (2, 2, 1) | value 42 | berat 3 | fragile False | ETA 4
G: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3
H: di luar truk | orientasi XYZ | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2
I: (1, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3
J: di luar truk | orientasi XYZ | ukuran (1, 4, 1) | value 45 | berat 4 | fragile True | ETA 1
K: di luar truk | orientasi XYZ | ukuran (3, 1, 2) | value 62 | berat 6 | fragile False | ETA 2
L: di luar truk | orientasi XYZ | ukuran (2, 1, 2) | value 43 | berat 4 | fragile False | ETA 1



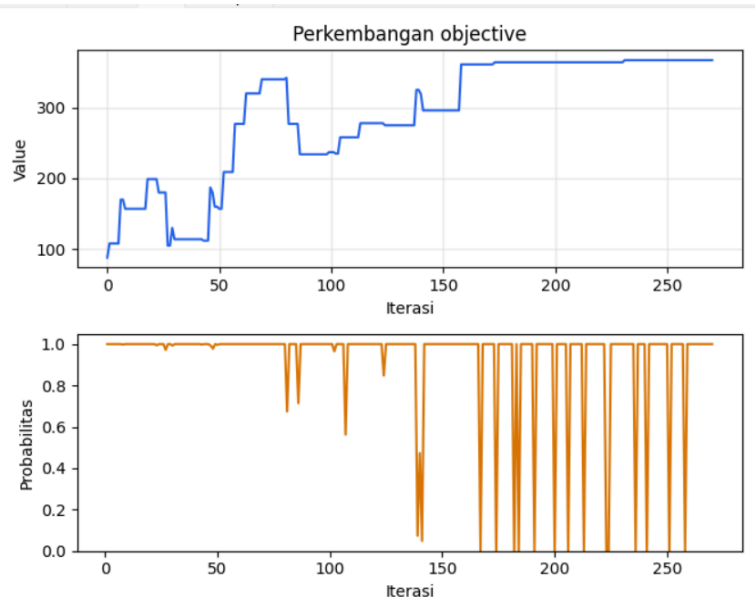
STATE AKHIR

A: (3, 1, 0) | orientasi ZYX | ukuran (1, 3, 3) | value 70 | berat 6 | fragile False | ETA 2
B: (2, 0, 0) | orientasi ZXY | ukuran (1, 2, 2) | value 65 | berat 5 | fragile True | ETA 3
C: (1, 1, 0) | orientasi ZYX | ukuran (1, 3, 1) | value 50 | berat 4 | fragile False | ETA 1
D: (1, 0, 1) | orientasi YXZ | ukuran (1, 2, 2) | value 52 | berat 4 | fragile False | ETA 2
E: (1, 2, 1) | orientasi XYZ | ukuran (1, 1, 1) | value 20 | berat 1 | fragile True | ETA 1
F: (0, 0, 0) | orientasi ZXY | ukuran (1, 2, 2) | value 42 | berat 3 | fragile False | ETA 4
G: di luar truk | orientasi YZX | ukuran (2, 2, 2) | value 75 | berat 7 | fragile False | ETA 3
H: di luar truk | orientasi YZX | ukuran (2, 2, 2) | value 88 | berat 9 | fragile True | ETA 2
I: (2, 2, 0) | orientasi ZXY | ukuran (1, 2, 3) | value 68 | berat 6 | fragile False | ETA 3
J: di luar truk | orientasi YXZ | ukuran (4, 1, 1) | value 45 | berat 4 | fragile True | ETA 1
K: di luar truk | orientasi YZX | ukuran (1, 2, 3) | value 62 | berat 6 | fragile False | ETA 2
L: di luar truk | orientasi YZX | ukuran (1, 2, 2) | value 43 | berat 4 | fragile False | ETA 1



Algoritma: Simulated Annealing
Seed: 42
Objective: Total value
Skor objective awal → akhir: 88 → 367

Paket di truk: 7/12
Berat akhir: 29/80
Iterasi: 270
Durasi: 0.0666 detik



Berdasarkan hasil yang diberikan simulated annealing dapat memberikan hasil yang mendekati global optima dan lebih baik dari stochastic hill climbing. Simulated annealing bisa memberikan hasil yang lebih karena terdapat penggunaan temperatur yang memperbolehkan bad move sehingga dapat mengeksplorasi state yang lebih buruk sehingga tidak tersangkut di local optima. Hasil yang diberikan menjadi lebih complete dari steepest ascent dan stochastic. Durasi yang dibutuhkan bisa sama dengan stochastic atau lebih kecil jika temperatur habis terlebih dahulu bila ratio cooling yang lebih kecil

2.5.3. Genetic Algorithm

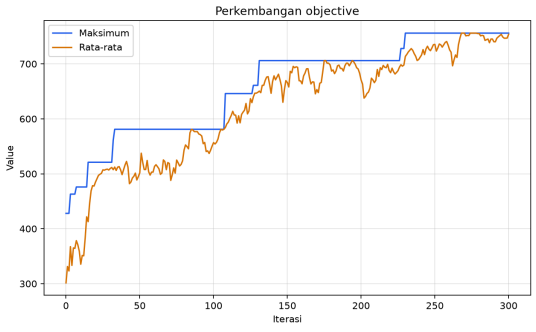
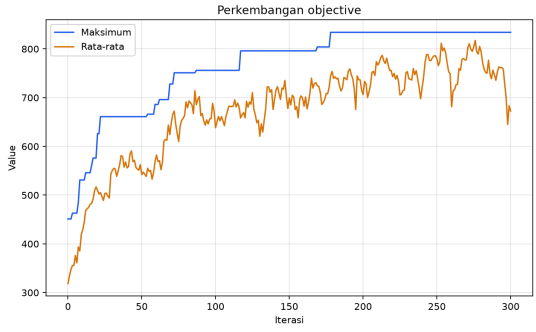
Eksperimen Genetic Algorithm dilakukan menggunakan dataset pada **test/GA.json.** Dataset tersebut terdiri atas 20 paket dan truk berukuran 6 × 8 × 4 dengan kapasitas berat maksimum 100. Objective yang digunakan adalah total value paket yang berada di dalam truk. Eksperimen dibagi menjadi dua kelompok. Kelompok pertama menggunakan populasi tetap sebanyak 20 individu dengan variasi iterasi 100, 200, dan 300. Kelompok kedua menggunakan iterasi tetap sebanyak 200 dengan variasi populasi 30, 60, dan 90 individu. Setiap konfigurasi dijalankan tiga kali menggunakan seed 67, 68, dan 69. Seluruh run menggunakan data truk dan paket yang sama. Hasil lengkap tersedia dalam **folder output/ga-specific/**.

Berikut ringkasan hasil dari 18 run yang telah dilakukan:

Populasi	Iterasi	Objective Seed 67	Objective Seed 68	Objective Seed 69	Rata-rata Objective	Rata-rata Durasi
20	100	581	756	765	700,67	0,88 s
20	200	706	834	831	790,33	1,85 s
20	300	756	834	831	807	2,98 s
30	200	758	811	823	797,33	3,02 s
60	200	874	879	866	873	6,74 s
90	200	816	844	853	837, 67	9,24 s

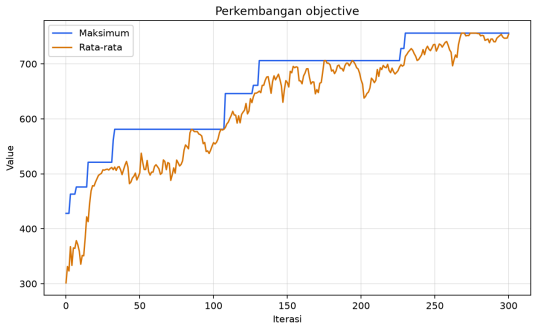
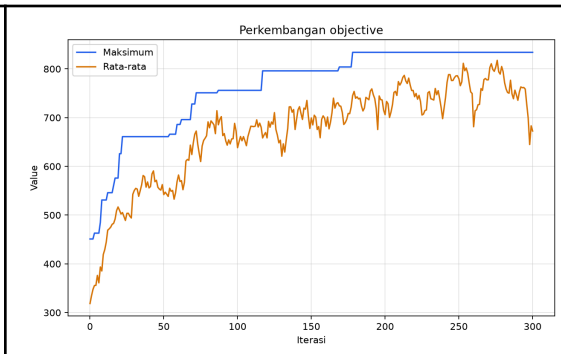
Pada kelompok variasi iterasi, penambahan iterasi dari 100 menjadi 200 meningkatkan nilai objective akhir pada ketiga seed. Rata-rata hasilnya meningkat dari 700,67 menjadi 790,33. Penambahan iterasi menjadi 300 masih meningkatkan rata-rata hasil, tetapi peningkatannya lebih kecil, yaitu menjadi 807,00. Berdasarkan result.json, seed 67 mencapai nilai 581 pada generasi 33 dan bertahan pada nilai tersebut sampai generasi 100. Setelah pencarian dilanjutkan, nilai maksimum meningkat menjadi 706 pada generasi 131 dan mencapai 756 pada generasi 230. Sebaliknya, seed 68 sudah mencapai nilai 834 pada generasi 178 dan tidak mengalami peningkatan lagi sampai generasi 300. Seed 69 juga tidak mengalami peningkatan setelah mencapai nilai 831 pada generasi 199. Hasil ini menunjukkan bahwa tambahan iterasi masih dapat membantu pencarian, tetapi manfaatnya berbeda pada setiap run. Dari 200 menjadi 300 iterasi, rata-rata objective hanya meningkat sekitar 2,1%, sedangkan durasinya meningkat sekitar 61,5%. Pada kelompok ini, variasi 200 iterasi memberikan hasil yang cukup baik dengan waktu pencarian lebih singkat daripada variasi 300 iterasi.

Contoh grafik eksperimen variasi iterasi:

<table>
<tr>
<td> 
</td>
<td> 
</td>
</tr>
<tr>
<td>populasi 20, iterasi 300, seed 67</td>
<td>populasi 20, iterasi 300, seed 67</td>
</tr>
</table>

Pada kelompok variasi populasi, penambahan populasi dari 30 menjadi 60 meningkatkan rata-rata objective akhir dari 797,33 menjadi 873,00. Namun, penambahan populasi menjadi 90 justru menurunkan rata-rata hasil menjadi 837,67. Populasi 60 menghasilkan nilai akhir lebih tinggi daripada populasi 90 pada ketiga seed. Perbedaannya juga terlihat pada riwayat fitness seed 68. Populasi 60 mencapai nilai 879 pada generasi 131, sedangkan populasi 90 masih mengalami peningkatan sampai generasi 180 tetapi hanya mencapai nilai 844. Jadi, populasi yang lebih besar dan pencarian yang masih mengalami peningkatan pada generasi akhir belum tentu menghasilkan solusi lebih baik. Populasi 60 juga memberikan hasil paling konsisten dengan rentang nilai 866-879. Sebagai perbandingan, populasi 30 menghasilkan rentang 758–823 dan populasi 90 menghasilkan rentang 816-853. Dari sisi waktu, populasi 60 membutuhkan durasi 6,74 detik yang lebih singkat daripada populasi 90 yang membutuhkan 9,24 detik.

Contoh grafik eksperimen variasi populasi:

	
populasi 20, iterasi 300, seed 67	populasi 20, iterasi 300, seed 67

Terdapat beberapa hal yang perlu diperhatikan dalam analisis eksperimen ini yaitu seed yang sama menghasilkan mutation rate yang sama untuk setiap konfigurasi, tetapi penggunaan seed berbeda dapat mengubah peluang mutasi yang berarti hal ini dapat berpengaruh dalam hasil eksperimen. Selain itu, setiap konfigurasi juga baru dijalankan tiga kali sehingga jumlah tersebut masih terbatas untuk menilai konsistensi hasil algoritma. Seluruh eksperimen juga dijalankan menggunakan satu dataset. Pengujian dengan lebih banyak pengulangan dan variasi dataset dapat membantu untuk melihat apakah pola hasilnya tetap sama.

Berdasarkan eksperimen yang dilakukan, konfigurasi populasi 60 dan 200 iterasi menghasilkan rata-rata objective akhir tertinggi, yaitu 873, serta selisih hasil terkecil pada tiga pengulangan. Penambahan iterasi membantu memperbaiki solusi, tetapi tambahan waktu pencarian tidak selalu diikuti peningkatan hasil yang besar. Selain itu,

penambahan populasi dapat menghasilkan rata-rata nilai fitness akhir yang lebih buruk.

Untuk eksperimen perbandingan dengan algoritma lain menggunakan test/tc1.json, tc2.json, dan tc3.json dengan populasi 60 dan iterasi 200 kali, didapatkan hasil sebagai berikut yang disimpan di **output/ga-general**:

Kasus	Objektif awal	Objektif akhir	Durasi
tc1.json	356	424	3,99 s
tc2.json	315	553	4,25 s
tc3.json	278	505	5,04 s

Hasilnya genetic algorithm mendapatkan nilai objektif akhir tertinggi sebesar 553 dengan rata-rata objektif akhir sebesar 494.

Untuk eksperimen konsistensi nilai akhir objektif menggunakan test/tc1.json yang dijalankan sebanyak 5 kali, didapatkan hasil sebagai berikut yang disimpan di **output/ga-consistency** dengan populasi 60 dan iterasi 200 kali:

Run	Objektif awal	Objektif akhir	Durasi
1	356	424	3,95 s
2	356	424	3,97 s
3	356	424	4 s
4	356	424	4 s
5	356	424	4 s

Hasilnya genetic algorithm konsisten dalam objektif awal dan akhir hanya perbedaan sedikit di durasi tiap run.

3. Kesimpulan dan Saran

3.1. Kesimpulan

3.1.1. Hill Climbing

1. Nilai Objective dari Hill Climbing terpengaruh langsung dari jenis algoritmanya, Steepest Ascent selalu meningkat nilai objectivenya, Sideways move selalu naik tetapi kadang terdiam jika di flat, Random

Restart nilai objective selalu naik kecuali ketika restart dan mungkin hasil akhir dari tiap restart lebih tinggi atau tiap rendah, Stochastic memberikan selalu naik tetapi tidak selalu konsisten naik dengan nilai objective tertinggi
2. Durasi dari Algoritma steepest ascent dan sideways move tidak bisa terlalu diprediksi tetapi steepest ascent sering mendapatkan waktu yang lebih cepat karena selalu berhenti di local optimum dan berhenti, steepest ascent terdapat lebih lama karena terdapat kemungkinan bergerak pada flat dan dapat bertemu flat yang lebih tinggi, Random restart terpengaruh dari jumlah restart yang diperbolehkan, dan Stochastic hanya terpengaruh oleh jumlah iterasi maksimal

3.1.2. Simulated Annealing

1. Nilai Objective dari Simulated Annealing terpengaruh oleh temperatur dan state random yang didapatkan, Hal tersebut membuat value dari Simulated Annealing pada permulaan beragam dan dapat naik turun. Karena pengurangan jumlah temperature semakin lama kemungkinan untuk mengambil state dengan value rendah berkurang sehingga pada akhir akan menuju sebuah optimum
2. Durasi Simulated Annealing terpengaruh dari dua hal yaitu max_iterations yang ditentukan dan temperatur, hasil akhir bisa seperti stochastic hill climbing jika max_iterations lebih cepat dibanding dari pengurangan temperatur, dan sebaliknya bekerja seperti itu juga

3.1.3. Genetic Algorithm

1. Pengaruh varian iterasi: durasi naik dan objektif akhir naik secara tetap namun objektif akhir antara iterasi 200 dan 300 tidak berbeda jauh.
2. Pengaruh banyak populasi: durasi naik, tetapi rata-rata objektif turun di populasi 90 walaupun naik dari populasi 30 ke 60.

Berdasarkan eksperimen pada tiga test case, Genetic Algorithm menghasilkan rata-rata objective akhir tertinggi sebesar 494, diikuti Random Restart Hill Climbing sebesar 469,67, Sideways Move sebesar 432,67, Simulated Annealing sebesar 394, Steepest Ascent sebesar 393, dan Stochastic Hill Climbing sebesar 375. Hasil ini menunjukkan bahwa algoritma dengan kemampuan eksplorasi ruang solusi lebih luas cenderung memperoleh nilai objective yang lebih tinggi. Namun, keberhasilan setiap algoritma tetap bergantung pada konfigurasi awal, parameter pencarian, dan karakteristik persoalan.

Dari segi durasi, Stochastic Hill Climbing menjadi algoritma tercepat dengan rata-rata waktu 0,0183 detik, diikuti Simulated Annealing sebesar 0,0596 detik dan Steepest Ascent sebesar 0,1195 detik. Random Restart membutuhkan rata-rata 0,7252 detik, Sideways Move sebesar 1,0713 detik, sedangkan Genetic Algorithm memiliki durasi terlama sebesar 4,43 detik. Perbedaan ini dipengaruhi oleh mekanisme pencarian masing-masing algoritma. Algoritma yang mengevaluasi lebih banyak tetangga, melakukan restart, atau mengelola populasi membutuhkan waktu komputasi lebih besar.

Berdasarkan hasil eksperimen yang berada di **output/consistency/**, tiap algoritma menunjukkan hasil objektif akhir yang konsisten persis sama untuk konfigurasi yang identik.

3.2. Saran

Karena keterbatasan waktu dan sumber daya penulis, proyek ini masih bisa dikembangkan dan dieksplorasi lebih lanjut. Penulis menyarankan kepada pembaca untuk menambahkan perhitungan menggunakan multi truk dan melakukan eksplorasi dengan jenis search lainnya

4. Pembagian Tugas

Tugas	**NIM**
Steepest Ascent Hill-Climbing	13524129
Simulated Annealing	13524083
Genetic Algorithm	13524058
Integrasi	13524058, 13524129, 13524083
Core	13524058
Models	13524058
Hill-Climbing with Sideways Move	13524129, 13524083
Stochastic Hill-Climbing	13524129
Random Restart Hill-Climbing	13524129, 13524083
Objective Function Baru	13524058

5. Referensi

[1] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8.

[2] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk.

[3] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=aI41TGo9J9c.

6. Lampiran

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Muhammad Naufal Romi Annafi - 13524058

Mahmudia Kimdaro Amin - 13524083

Eduard Daniel Ariajaya - 13524129

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Kode Mata Kuliah : IF3170

Nama Mata Kuliah : Inteligensi Artifisial

Judul Tugas/Proposal : Tugas Besar 1 IF3170 Inteligensi Artifisial

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau	~~Ya~~/Tidak	

sejenisnya.		
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~	Menggunakan untuk membuat test case json
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~	Menggunakan untuk memastikan apa yang dibuat benar dan untuk mengerti apa yang perlu dilakukan
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak	
Lainnya, Jelaskan:		

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026

		
Muhammad Naufal Romi Annafi (13524058)	Mahmudia Kimdaro Amin (13524083)	Eduard Daniel Ariajaya (13524129)